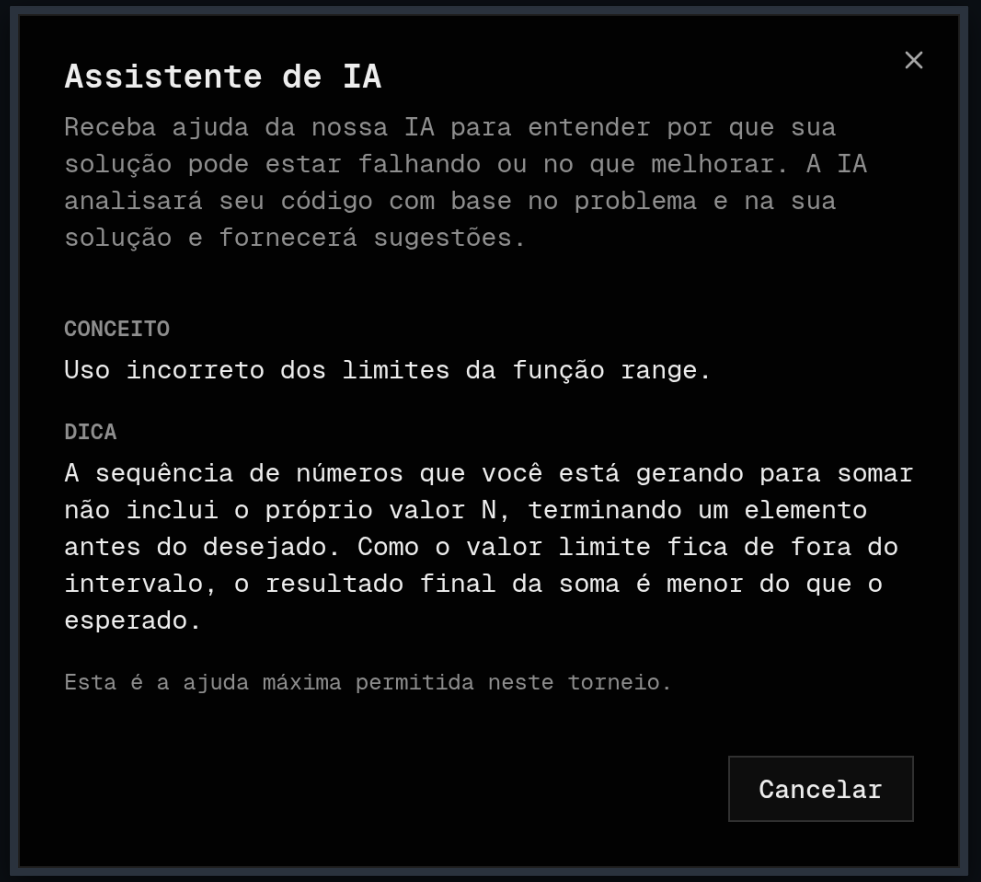
$ ./whyrunner --tcc-ii --v4

WhyRunner

Juiz online para o ensino de programação, com dicas de IA que o professor controla

Juan Israel dos Anjos
Orientadora: Suellen Mapa
Bacharelado em Sistemas de Informação · IFMG Ouro Branco
TCC II · Apresentação · outubro de 2026



Dica da IA em dois níveis, dentro de um torneio

Juízes online corrigem código; ensinar é outra coisa

✗ Wrong Answer

caso de teste 3 de 10 · sem explicação

Feedback e atividades curadas tornam o juiz útil para ensinar?

PERGUNTA DE PESQUISA

Feedback por IA e atividades curadas tornam o juiz útil para ensinar?

O objetivo é um juiz online feito para a sala de aula

executa com segurança

explica o erro

professor no controle

Nenhum dos sistemas analisados ensina, só corrige

Sistema	Feedback IA	Linguagens	Tempo real	Aulas curadas	Restrições	Portugol + blocos
Ataux (2021)	—	Limitadas	—	—	—	—
GOJ – UnB (2022)	—	Moderadas	Limitado	—	—	—
Run Codes	—	Muitas	●	—	—	—
BOCA (2004)	—	Moderadas	●	—	—	—
WhyRunner	●	**6**	●	●	●	●

Diferencial: dicas por IA, aulas montadas pelo professor, Portugol com blocos e restrições de solução: verificar *como* o aluno resolveu, não só se a saída está certa.

Três coisas que nenhum dos sistemas analisados tem

dicas por IA

aulas do professor

Portugol com blocos

Ciclos iterativos: cada versão corrigiu a anterior

	TENTADO		ADOTADO	POR QUÊ
Fila	~~BullMQ → AWS SQS~~	→	**PostgreSQL LISTEN/NOTIFY**	sem polling ocioso nem outro serviço pago
Backend	~~API NestJS separada~~	→	**Next.js Server Actions**	tipos e autenticação não ficam duplicados
Banco	~~Supabase~~	→	**PostgreSQL + Drizzle**	liberdade para lógica no servidor
Sandbox	~~Docker por submissão~~	→	**isolate (usado na IOI; MAREŠ; BLACKHAM, 2012)**	Docker exigia host privilegiado

Levantamento: literatura + conversas com alunos + experiência própria

MVP testado informalmente no Discord

Dois serviços que só conversam pelo banco



O site grava; o judge escuta, executa e grava o veredito.

A IA passa por uma cadeia de três modelos



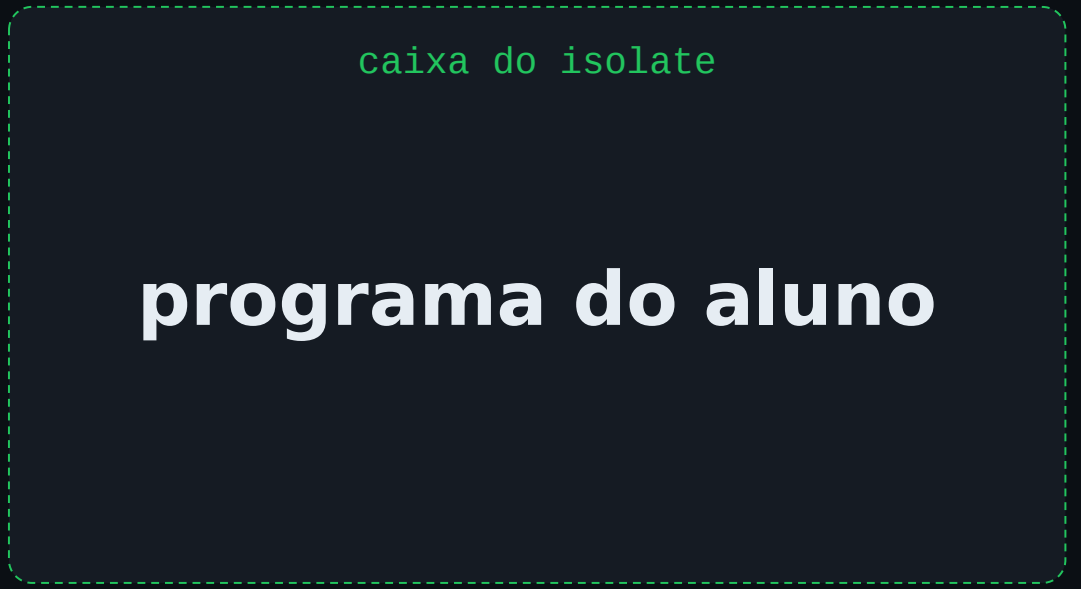
30 s por tentativa · só o Next.js chama a IA

O judge executa e corrige; o site só pede

O judge

execução, veredictos e comparação

Todo código roda numa caixa sem rede



sem rede

tempo e memória

≤ 64 processos

Seis linguagens, do C ao Portugol

C

C++

Java

Python

Rust

Portugol

Cada falha agora tem nome: seis veredictos

AC	WA	TLE	MLE	CE	RE
Aceito	Resposta errada	Tempo excedido	Memória excedida	Erro de compilação	Erro de execução

Antes: só “passou” ou “falhou”.

Aceito: passou em todos os casos de teste



Passou em todos os casos de teste

veredito: PASSED

Resposta errada: a saída não bate com a esperada



A saída não bate com a esperada

esperado 15 · recebido 14

Tempo excedido: o programa não terminou a tempo

TLE

Não terminou dentro do limite

limite do problema × fator da linguagem

Memória excedida: o programa pediu memória demais

MLE

Pediu mais memória que o limite

128 MB em C, C++ e Python

Erro de compilação: o código nem chegou a rodar

CE

O código não compilou

a saída do compilador é mostrada ao aluno

Erro de execução: o programa quebrou no meio

RE

O programa terminou com erro

divisão por zero · índice fora do vetor

O aluno vê o veredito de cada tentativa



Seis tentativas, seis vereditos

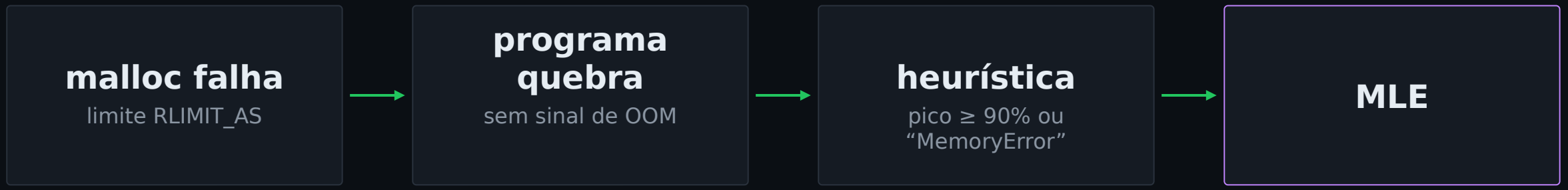
Erro de compilação não penaliza no placar

WA, TLE, RE...
+20 min por tentativa

CE
não conta no placar

Convenção da ICPC: quem não compilou não foi julgado.

Memória excedida é deduzida, não detectada



Não há sinal limpo de estouro: o judge deduz.

A saída pode ser comparada de quatro jeitos

exata
“3 4” ≠ “3 4”

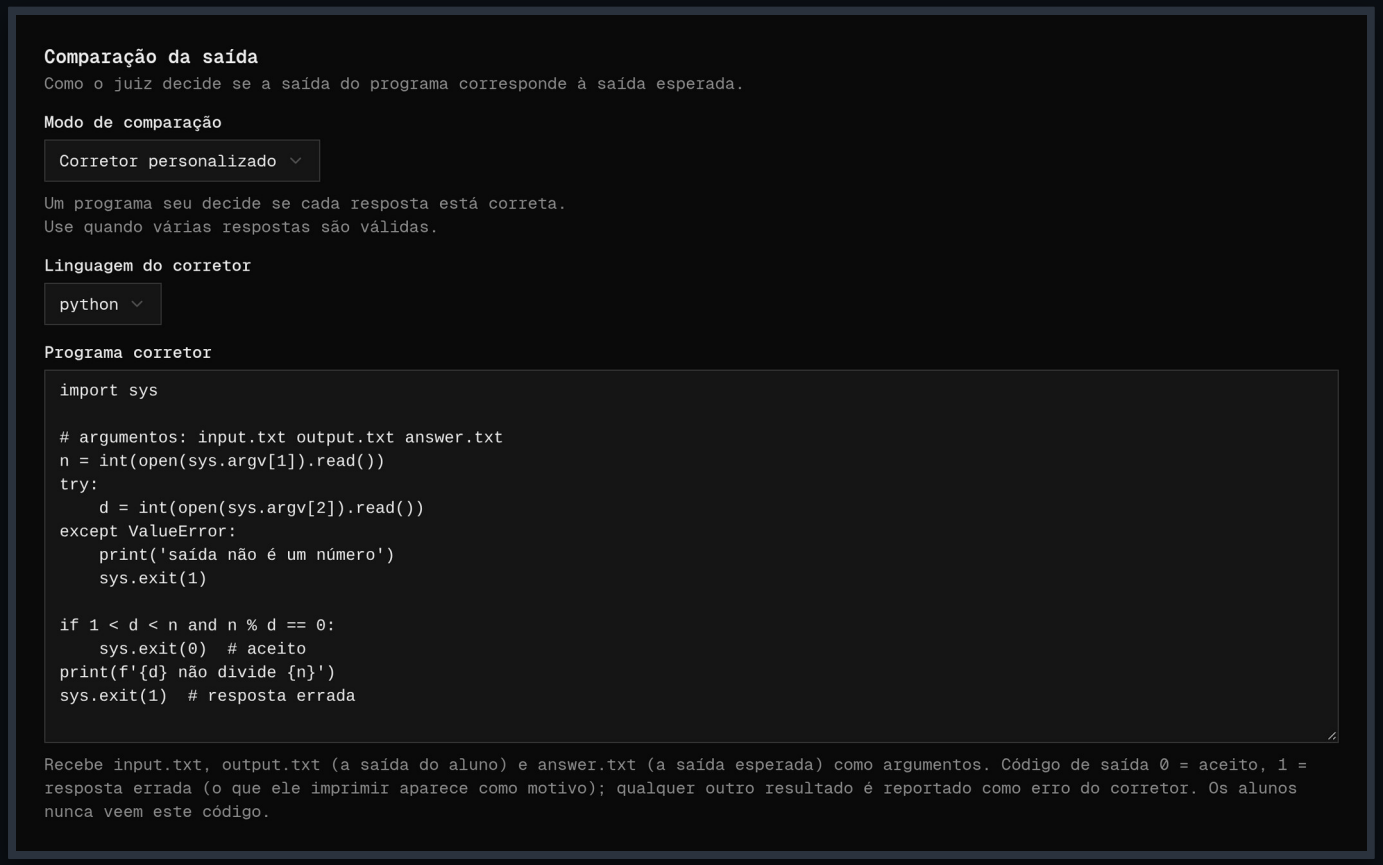
tokens
espaços não importam

float (ε)
3,1416 ≈ 3,14159

corretor
programa do professor

Problema novo começa em “tokens”.

Quando há várias respostas certas, um corretor decide



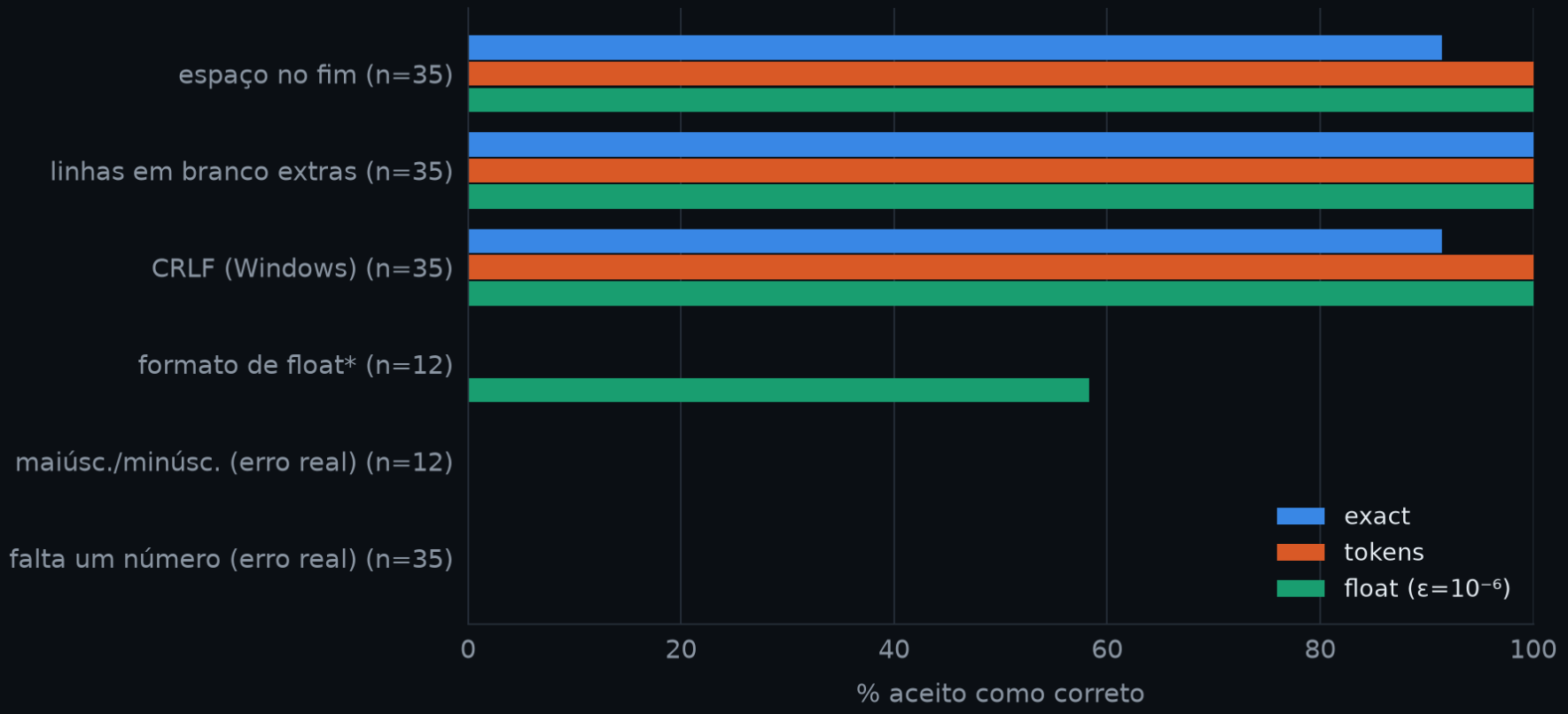
Problema: imprima qualquer divisor de N

O corretor recebe três arquivos e responde pelo código de saída



Roda no isolate, como o código do aluno.

Ignorar espaços aceita respostas certas que a comparação exata recusava



Saídas certas reescritas de jeitos comuns

Medimos o judge com benchmarks reproduzíveis

Desempenho

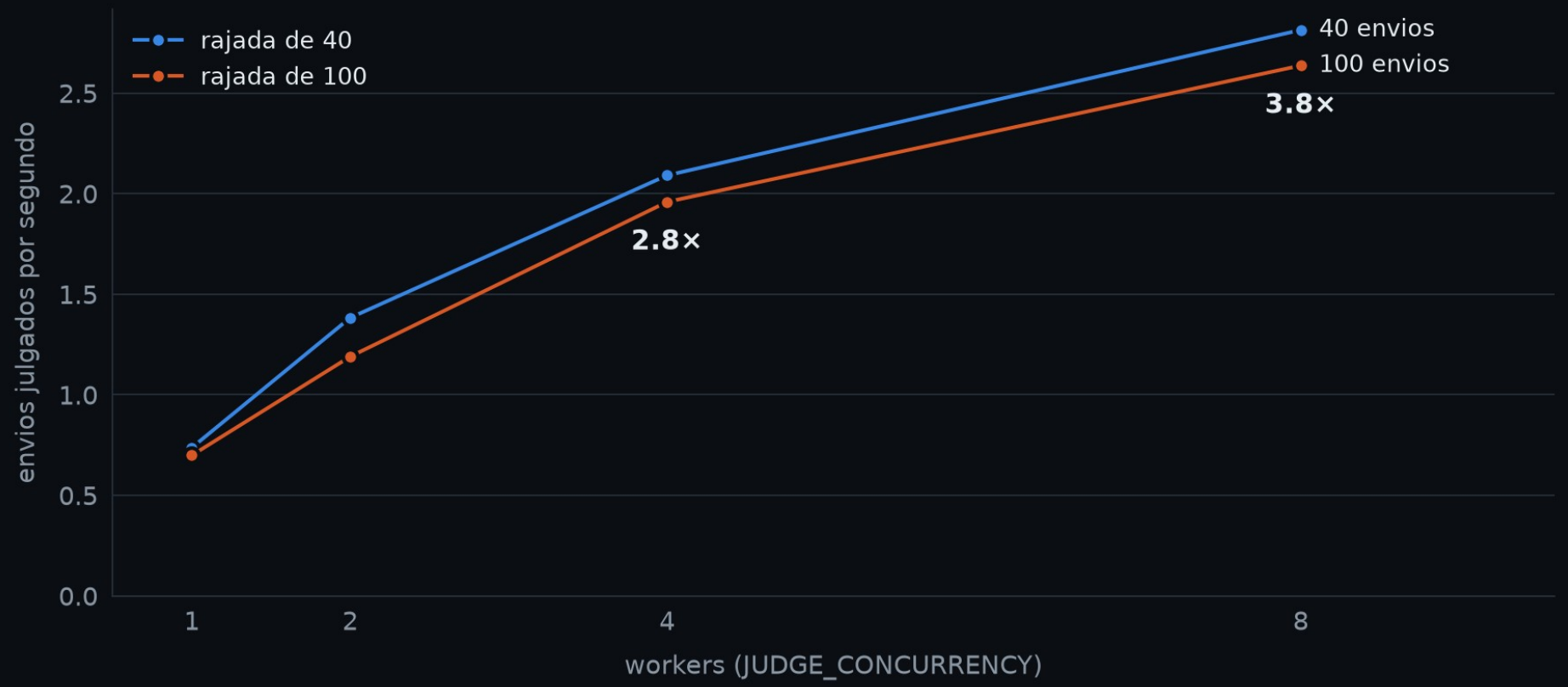
bench/ · dados brutos e scripts no repositório

O judge agora julga vários envios ao mesmo tempo



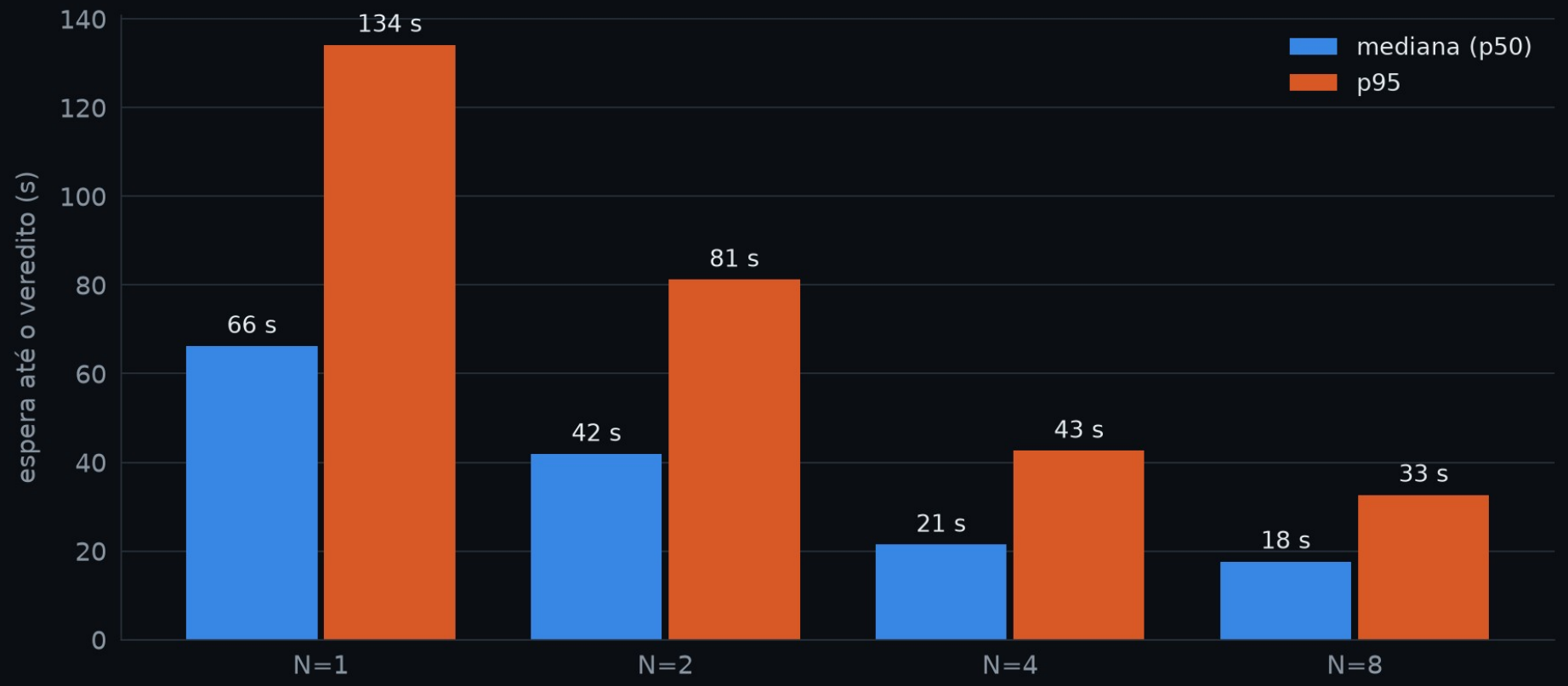
N = JUDGE_CONCURRENCY (padrão: nº de CPUs, máx. 4)

4 workers julgam 2,8× mais rápido; 8 workers, 3,8×



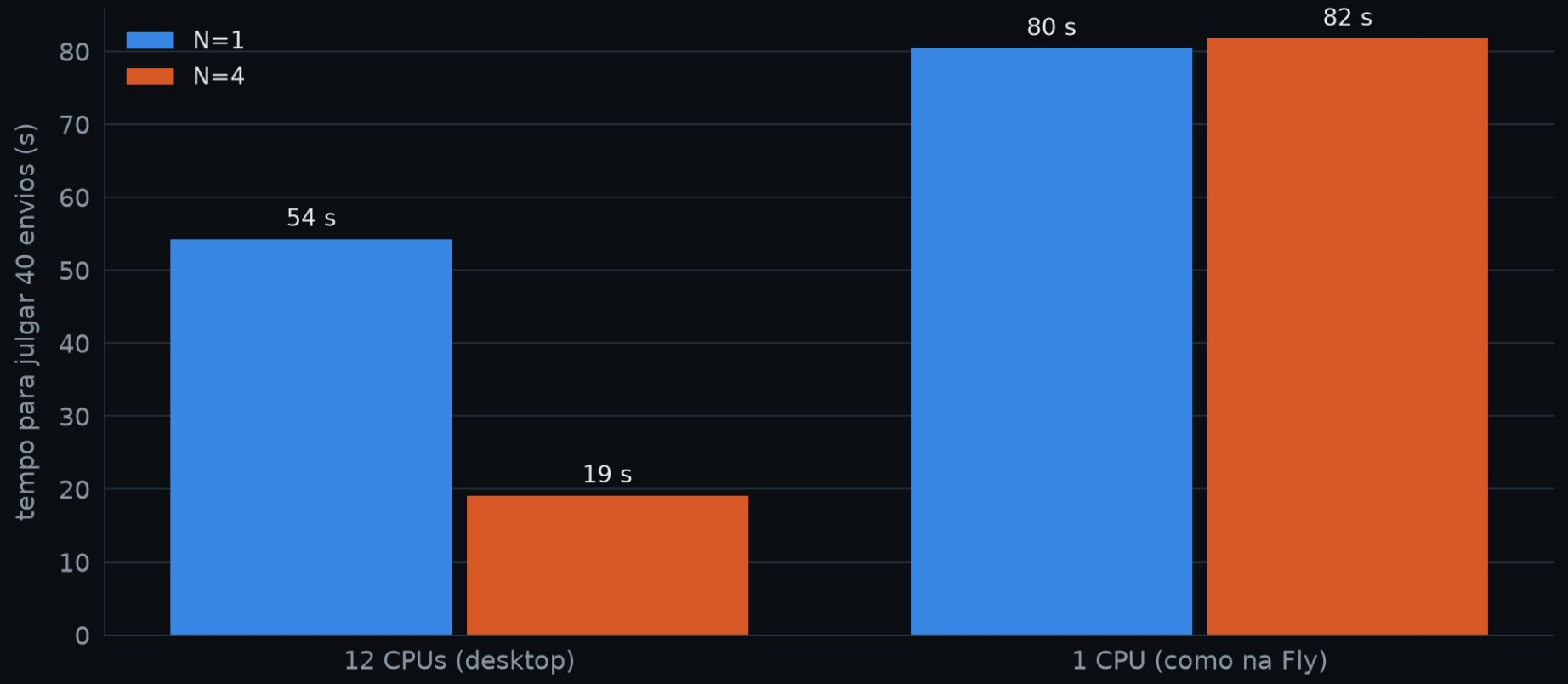
Rajada de envios nas 6 linguagens, 5 testes cada

Com 8 workers, metade da turma espera menos de 18 s, não 66 s



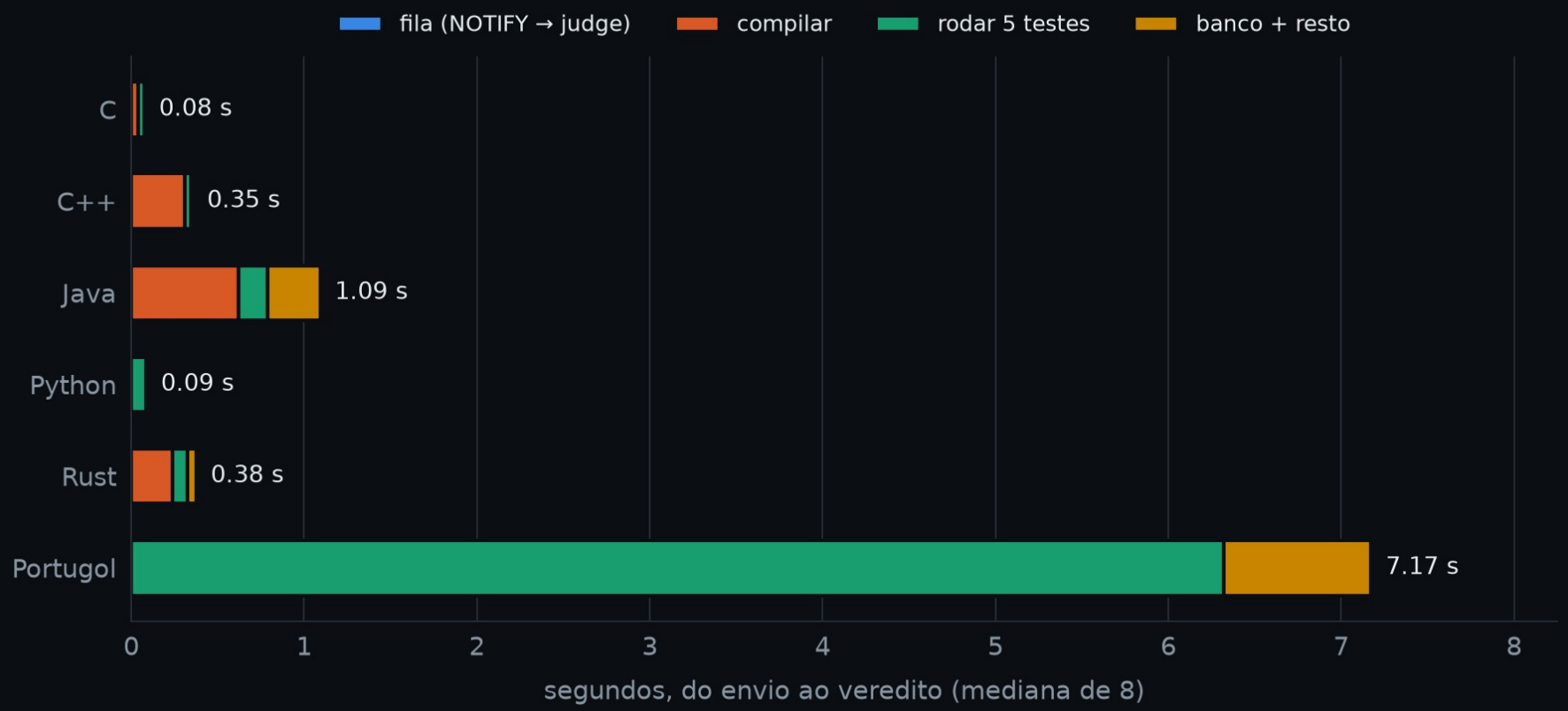
Rajada de 100 envios: espera até o veredito

Em 1 CPU, mais workers não ajudam, e a produção roda assim



Mesma rajada de 40, com o contêiner preso a 1 CPU

Sem fila, o veredito sai em menos de 0,4 s, exceto Java e Portugol



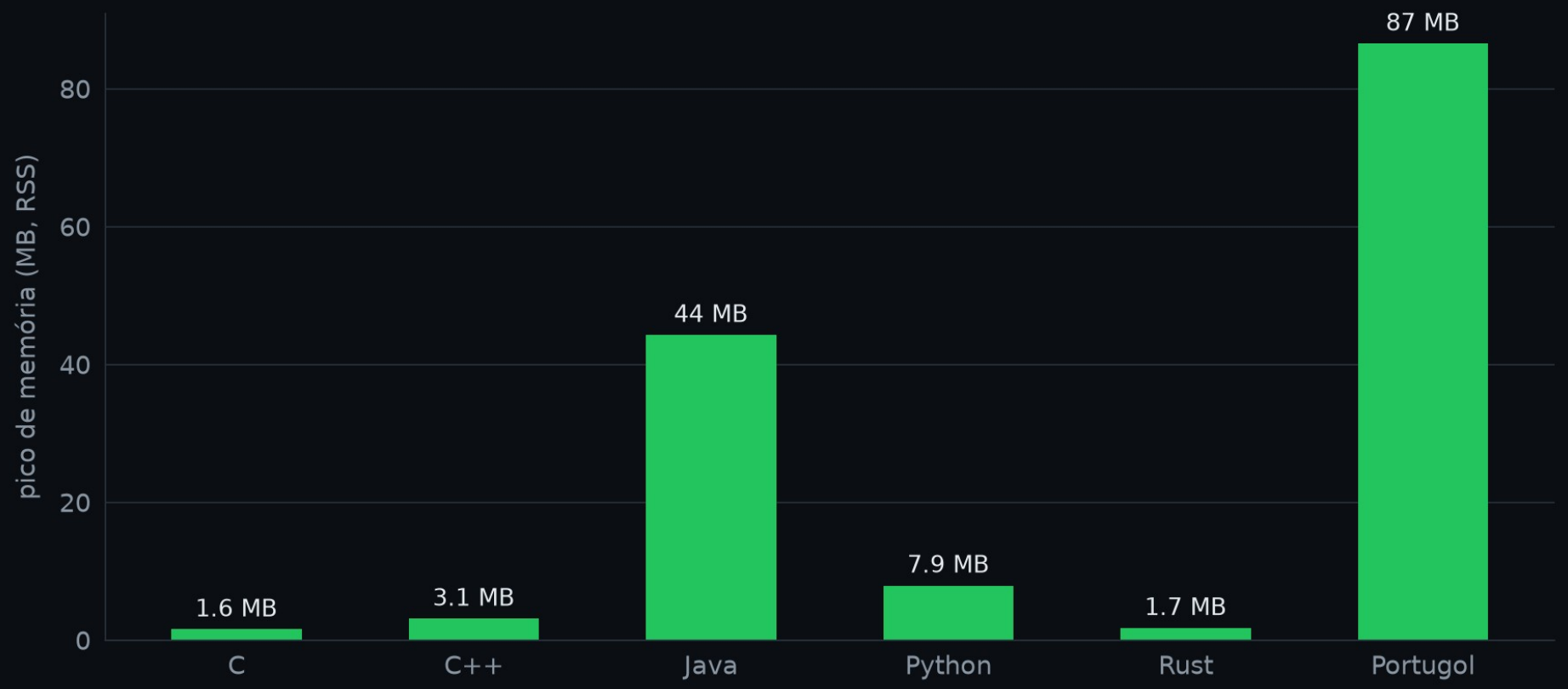
Um envio por vez, judge ocioso, 5 casos de teste

Compilar custa mais que rodar, exceto no Portugol



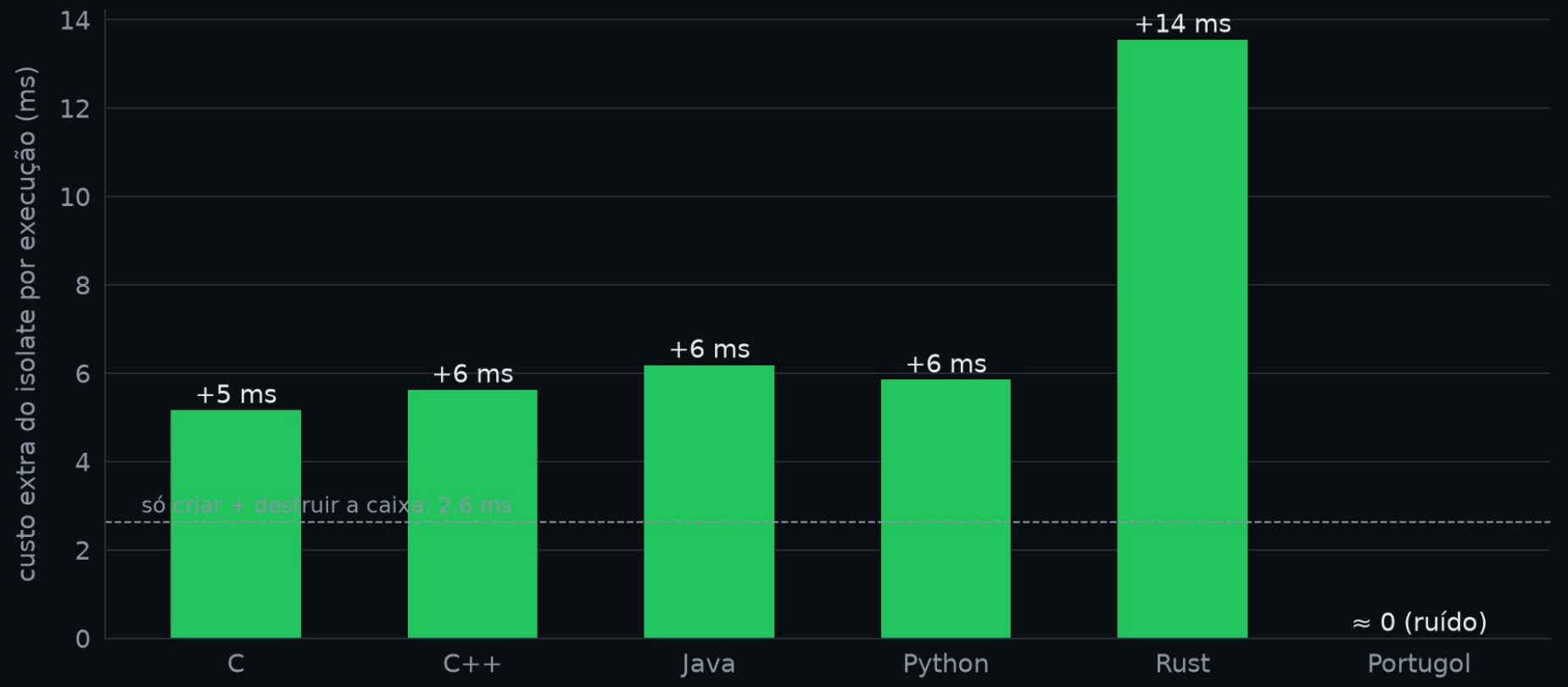
Mediana de 6 execuções no isolate

Java e Portugol usam dezenas de vezes mais memória



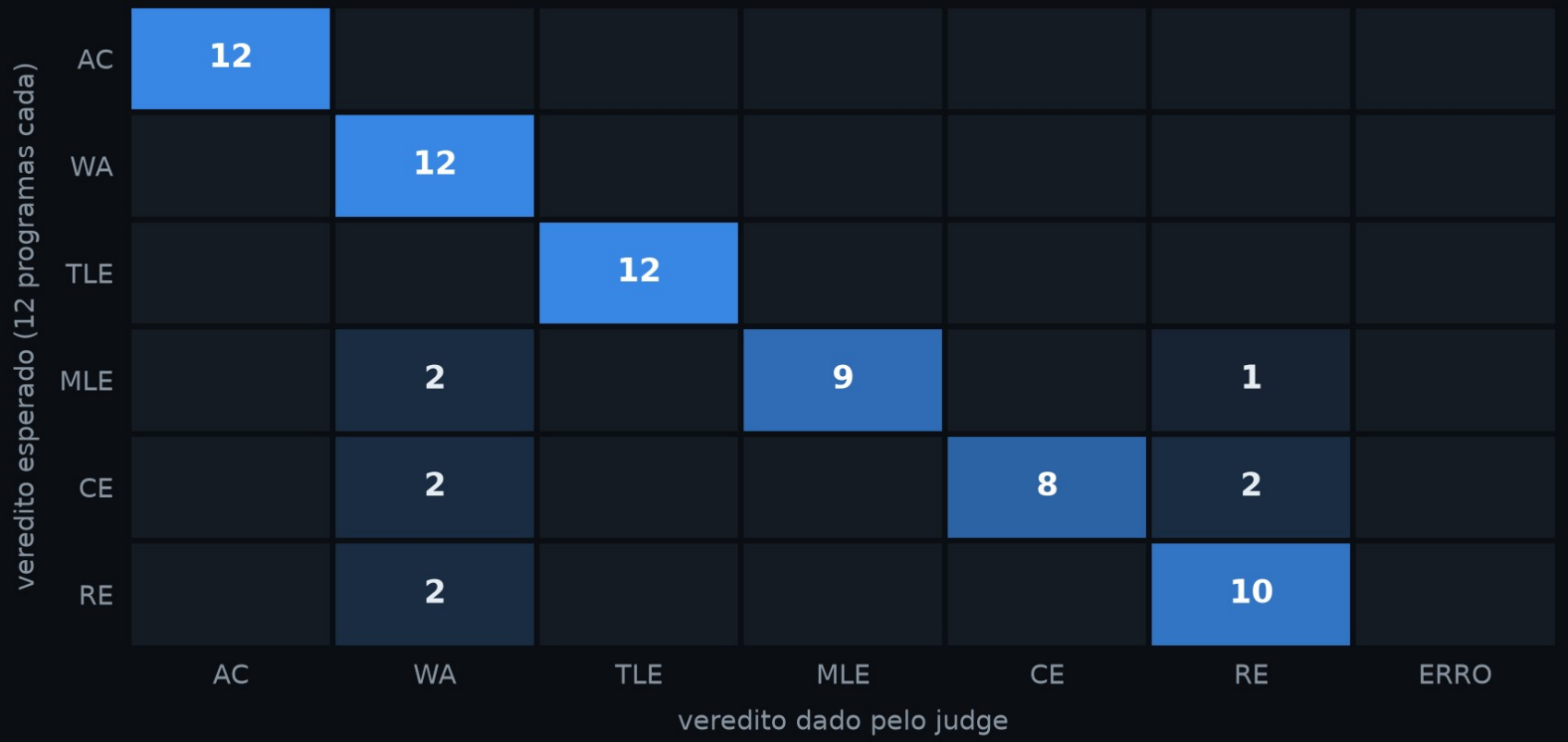
Pico de memória no laço de 10^7 passos

O isolate custa cerca de 5 ms por execução



Mesmo programa, com e sem caixa

O judge acertou 63 de 72 vereditos; os erros têm causa conhecida



2 programas por veredito × 6 linguagens

51 testes no judge, 112 no site

51

testes do judge

112

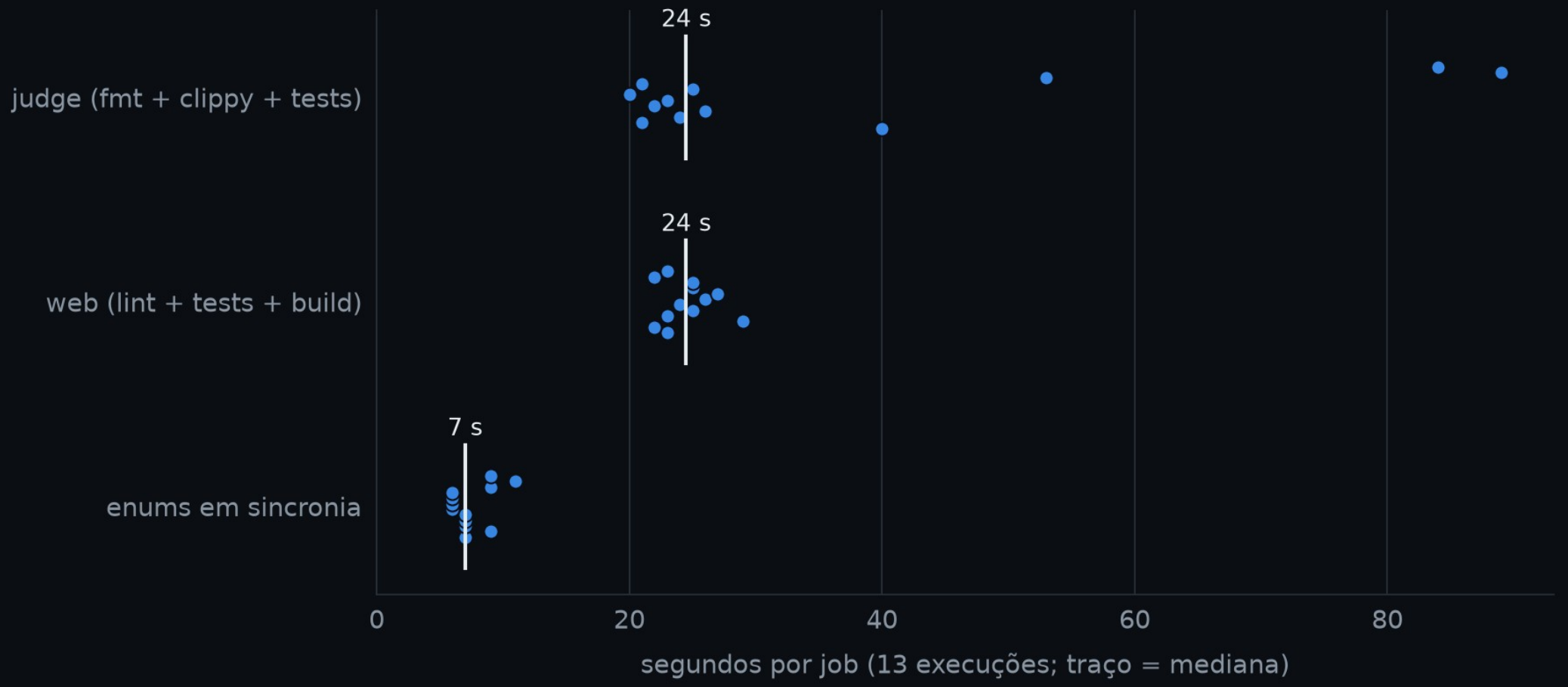
testes do site

Cada push roda três jobs no GitHub Actions



Os três rodam em paralelo.

O CI leva cerca de 31 s por push



Um script garante que Rust e TypeScript concordam nos enums



Os dois serviços dividem o banco, não o código.

O professor cria problemas, torneios e aulas

O professor

problemas · torneios · aulas

Nenhum problema é publicado sem uma solução que passa

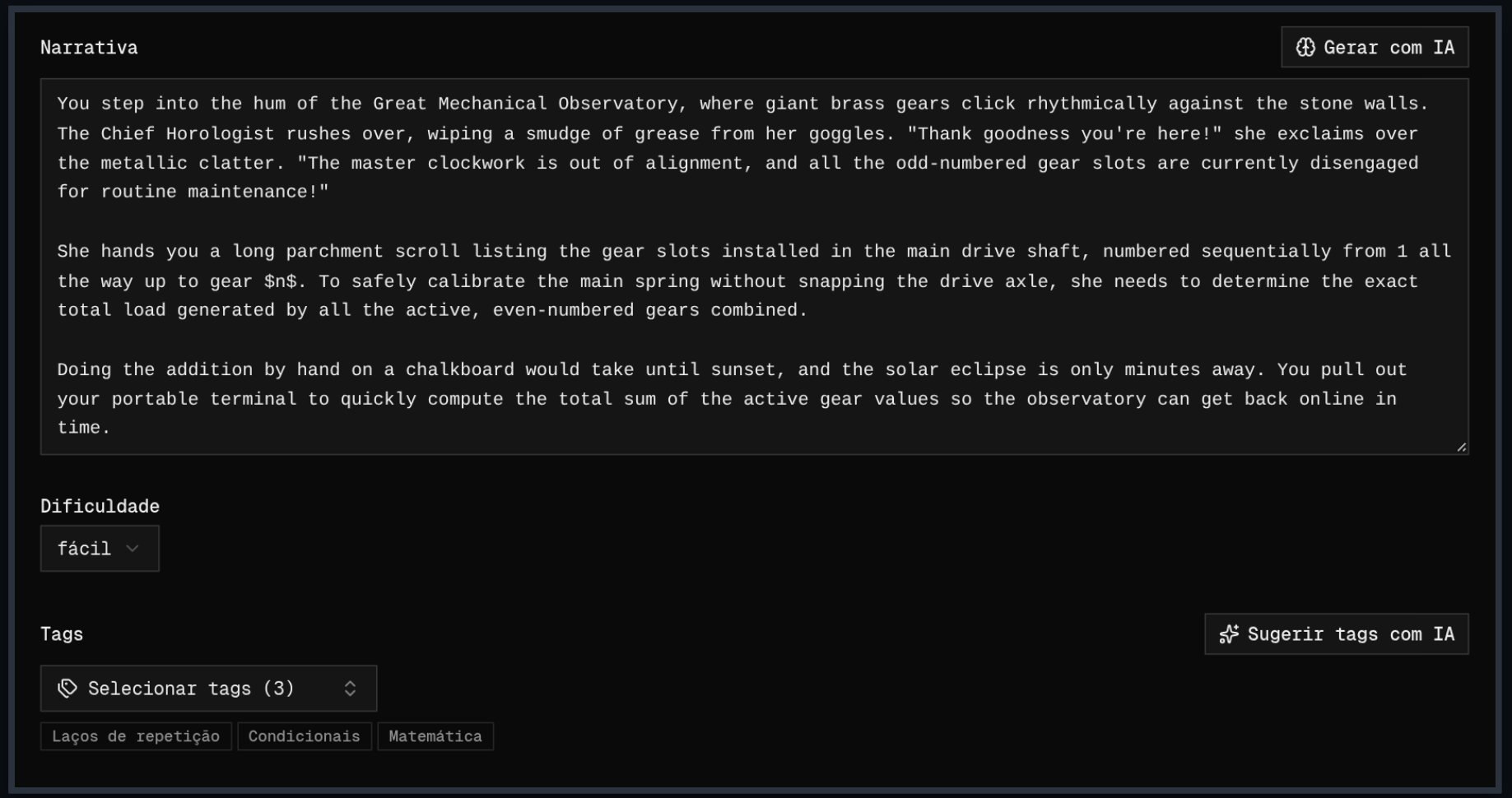


A IA escreveu soluções que passaram em 18 de 20 problemas

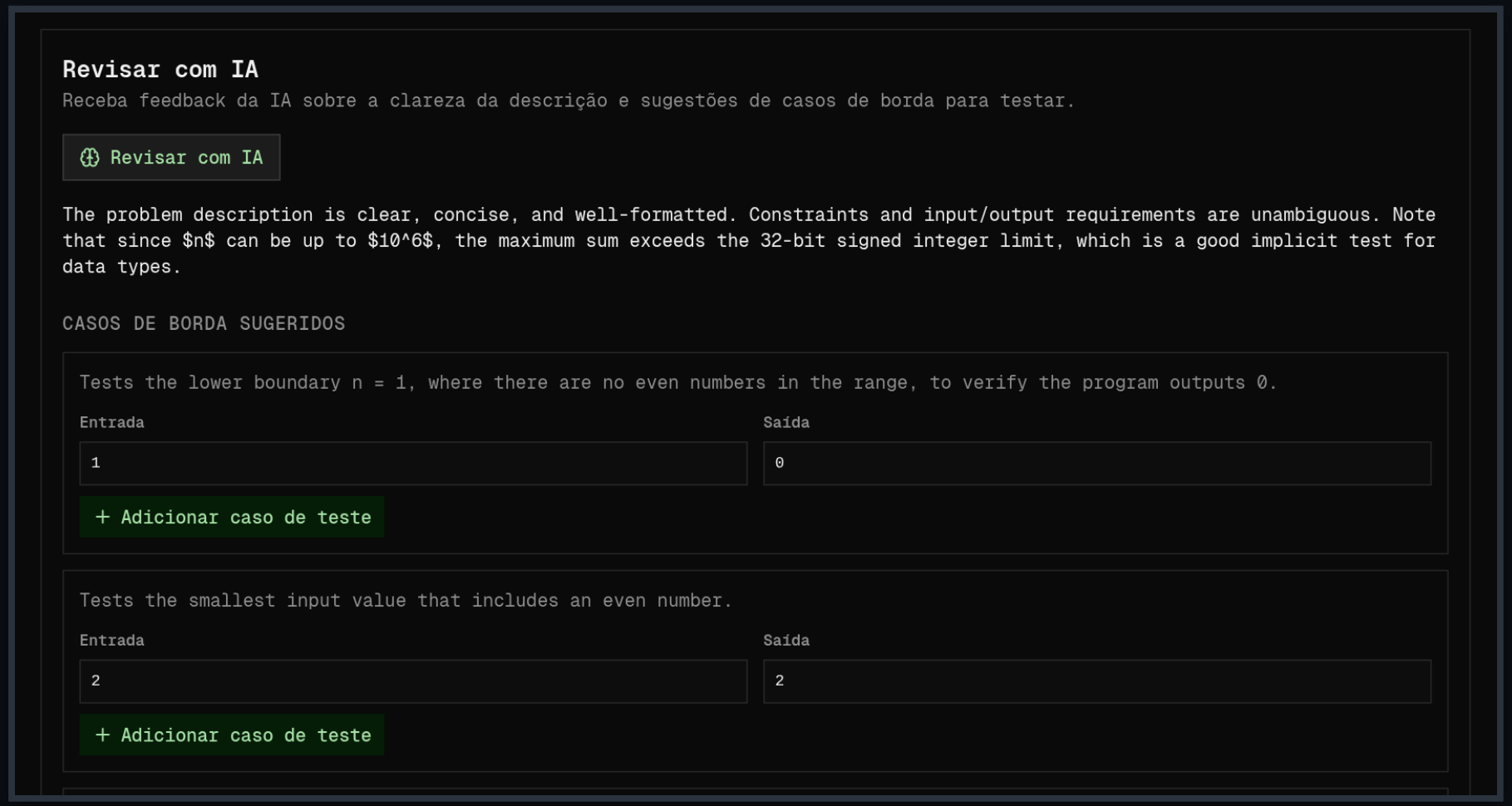


Soluções de referência geradas pela IA, julgadas pelo judge

A IA sugere a narrativa e as tags do problema



A IA aponta casos de borda que faltam nos testes



O professor vê o problema como o aluno vai ver

Soma dos pares

FÁCIL

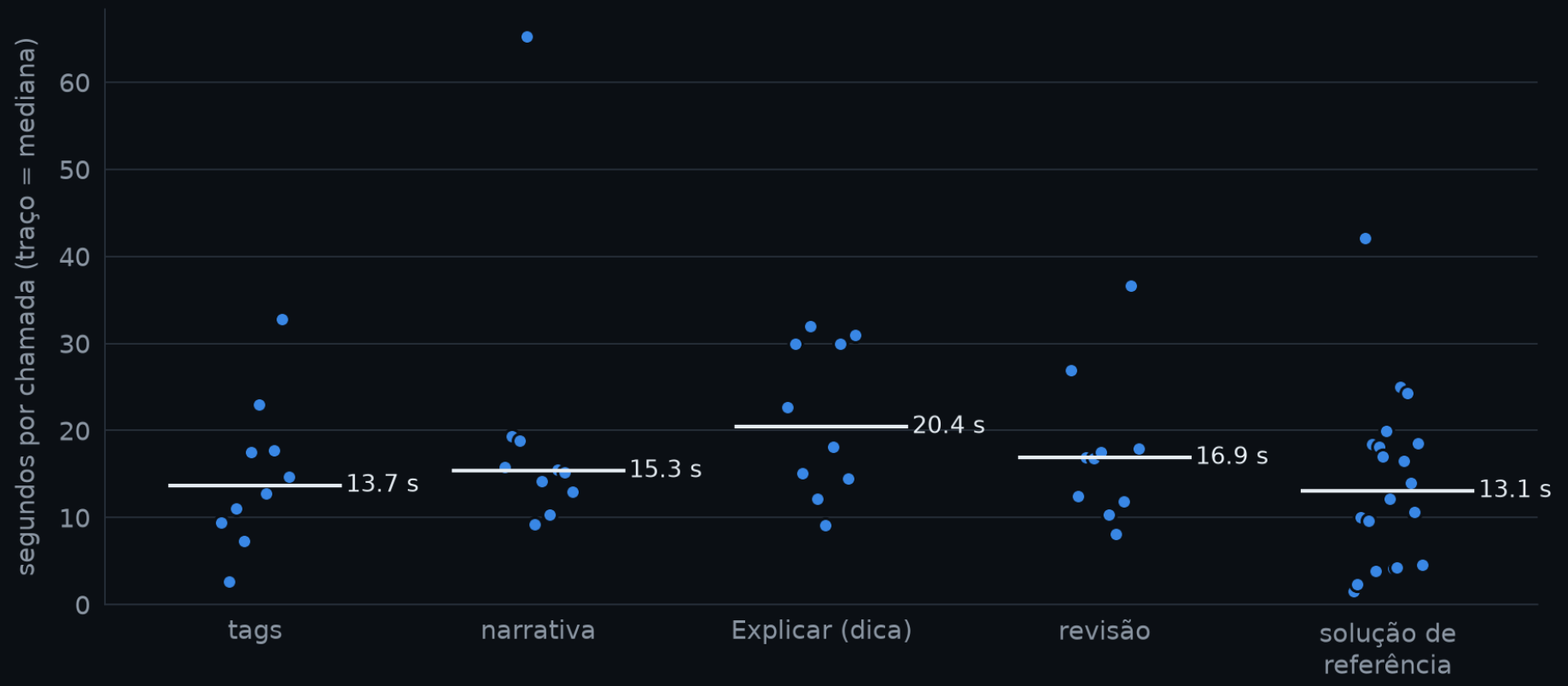
Laços de repetição | Condicionais | Matemática

NARRATIVA

You step into the hum of the Great Mechanical Observatory, where giant brass gears click rhythmically against the stone walls. The Chief Horologist rushes over, wiping a smudge of grease from her goggles. "Thank goodness you're here!" she exclaims over the metallic clatter. "The master clockwork is out of alignment, and all the odd-numbered gear slots are currently disengaged for routine maintenance!"
She hands you a long parchment scroll listing the gear slots installed in the main drive shaft, numbered sequentially from 1 all the way up to gear n. To safely calibrate the main spring without snapping the drive axle, she needs to determine the exact total load generated by all the active, even-numbered gears combined.
Doing the addition by hand on a chalkboard would take until sunset, and the solar eclipse is only minutes away. You pull out your portable terminal to quickly compute the total sum of the active gear values so the observatory can get back online in time.

Leia um inteiro positivo n e imprima a soma de todos os números pares entre 1 e n (inclusive).
Entrada: um inteiro n ($1 \leq n \leq 10^6$).
Saída: a soma dos pares de 1 a n.

Cada ajuda da IA leva de 13 a 20 s



10 chamadas por recurso (20 na solução de referência)

A cadeia de modelos salvou mais da metade das chamadas

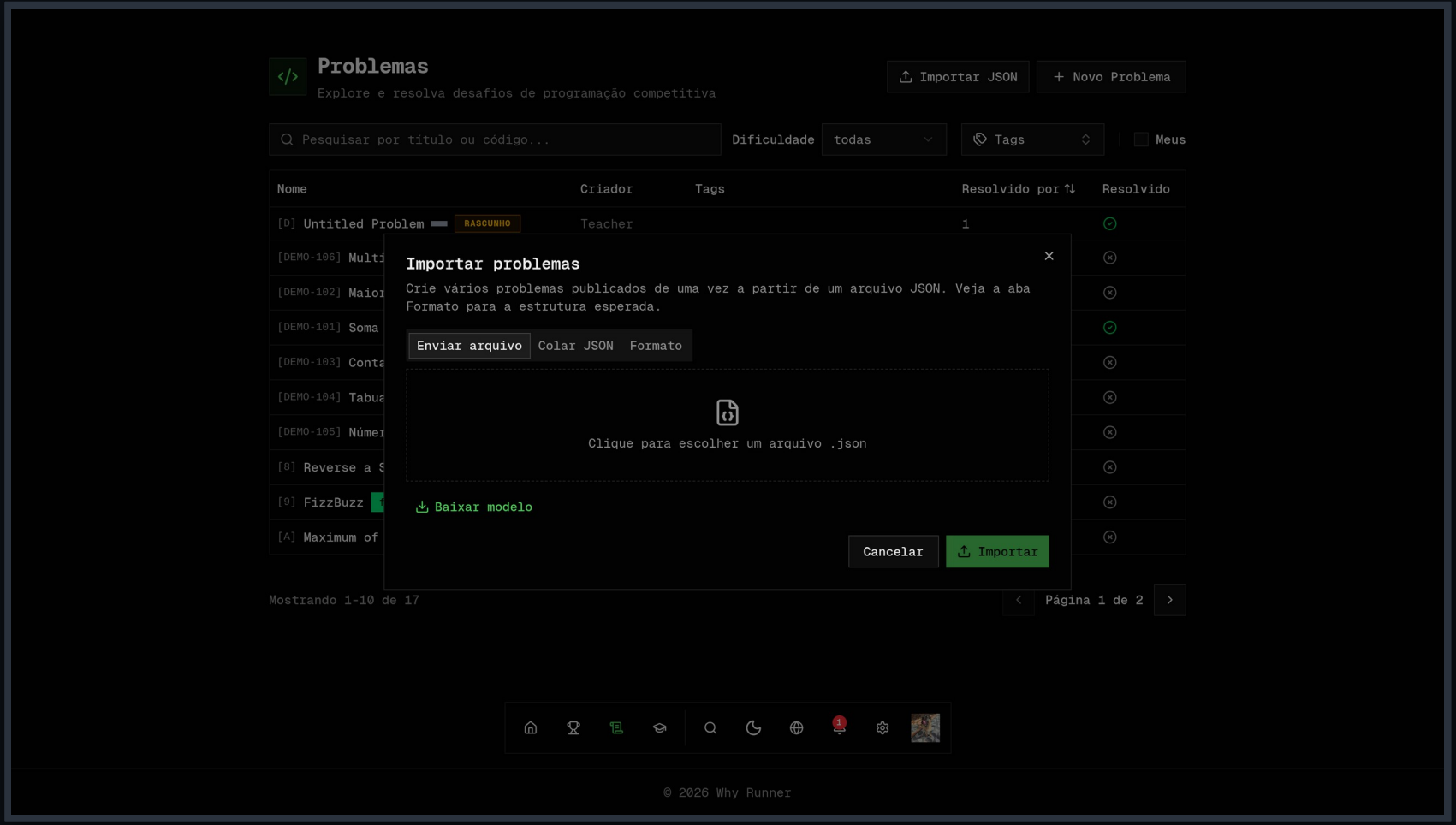


60 chamadas · 0 sem resposta

O banco de problemas filtra por código, tema e dificuldade



Vários problemas entram de uma vez por JSON



O torneio se configura em uma tela



Nome, datas, privado e nível de IA

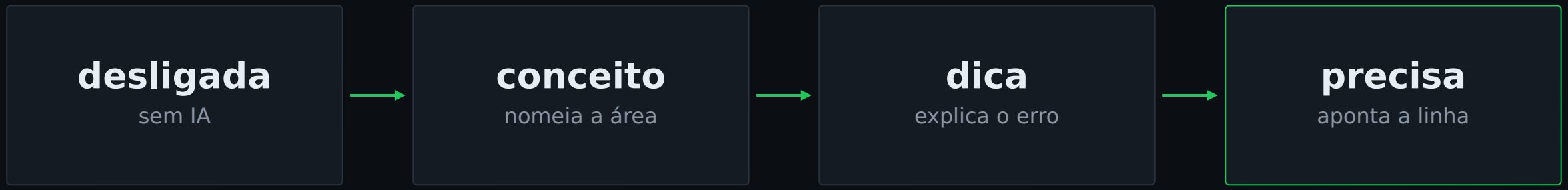
Problemas entram no torneio pelo código



No torneio privado, o professor aprova quem entra



O professor decide quanto a IA ajuda no torneio



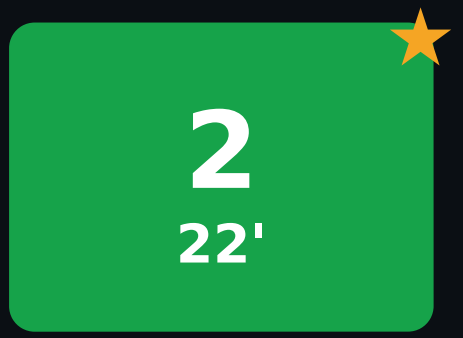
O aluno libera um nível por vez.

Placar no estilo das maratonas de programação

#	Participante	A	B	C	D	Resolvidos	Penalidade
1	LU Lucas Andrade	1 6'	2 22'	1 70'	2 104' ★	4	242
2	RA Rafael Souza	1 4' ★	1 12' ★	2 49' ★	-	3	85
3	BE Beatriz Lima	1 9'	1 30'	2 58'	1	3	117
4	 user	5 52'	1	-	-	1	132

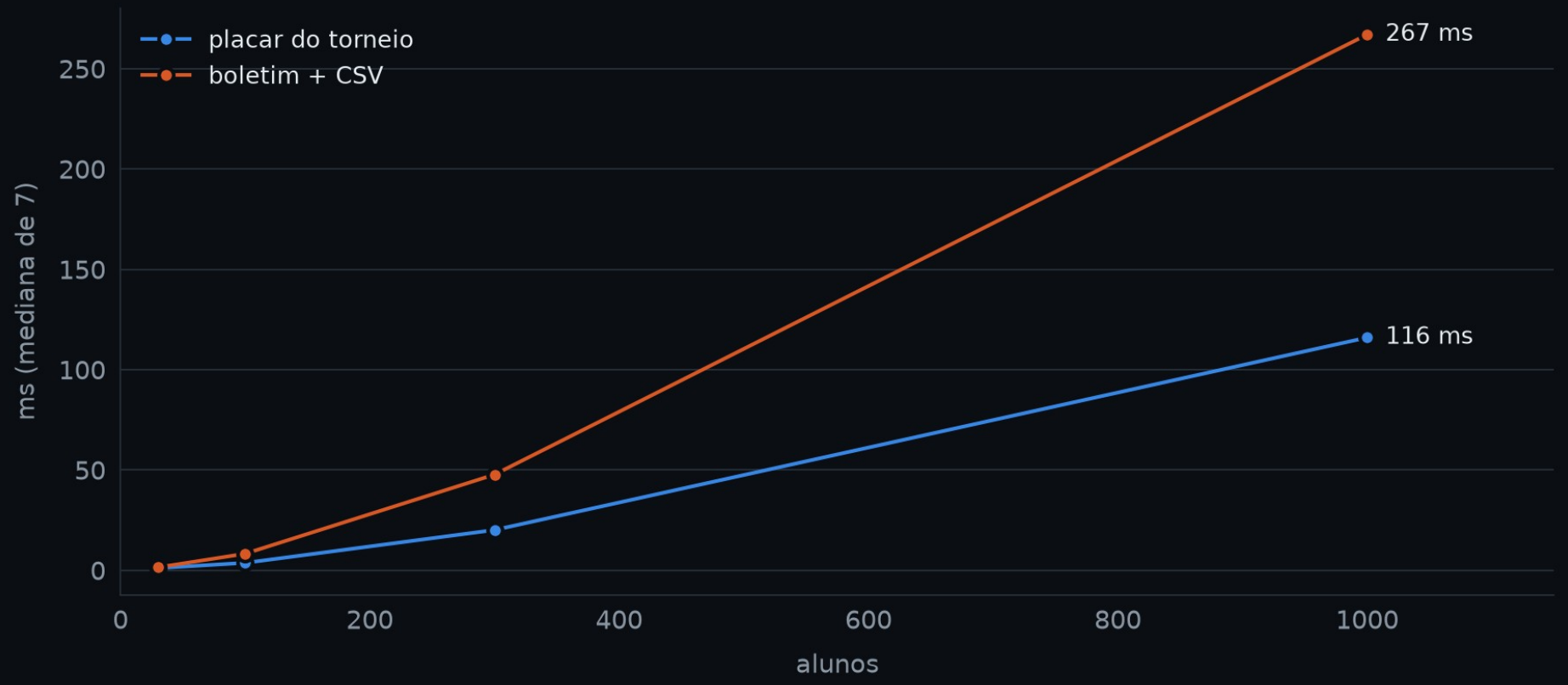
Número = tentativas · 34' = minuto da resolução · ★ = primeiro a resolver · Penalidade = minutos das resoluções + 20 por tentativa errada em problemas resolvidos

Cada célula conta tentativas, minuto e quem resolveu primeiro



2 tentativas
22' minuto do acerto
★ primeiro a resolver

Placar de 1000 alunos em 0,12 s; boletim em 0,27 s



Cálculo puro, sem banco, dados sintéticos

O aluno resolve, envia e entende o erro

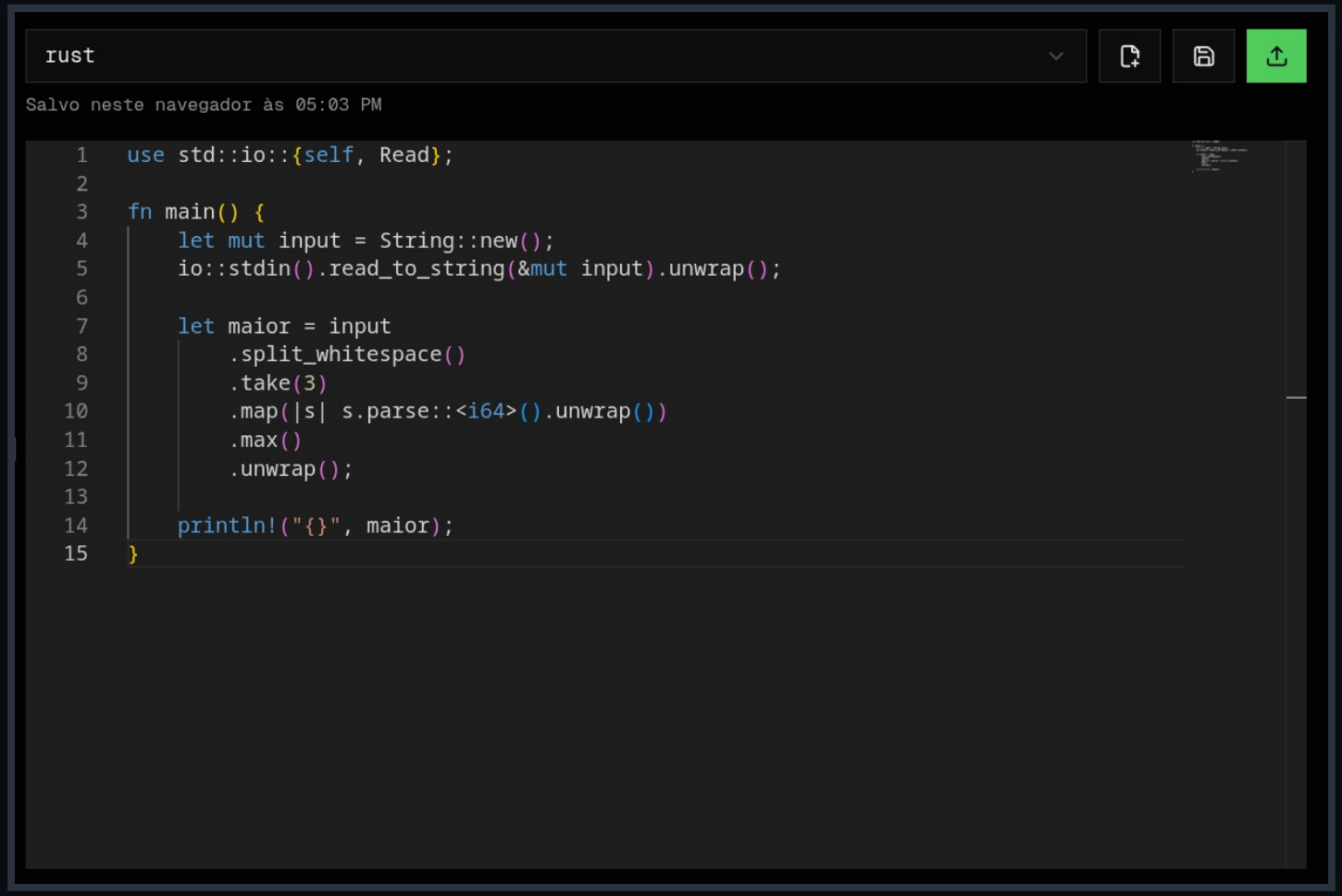
O aluno

resolver · entender · voltar

A tela inicial mostra o que falta fazer



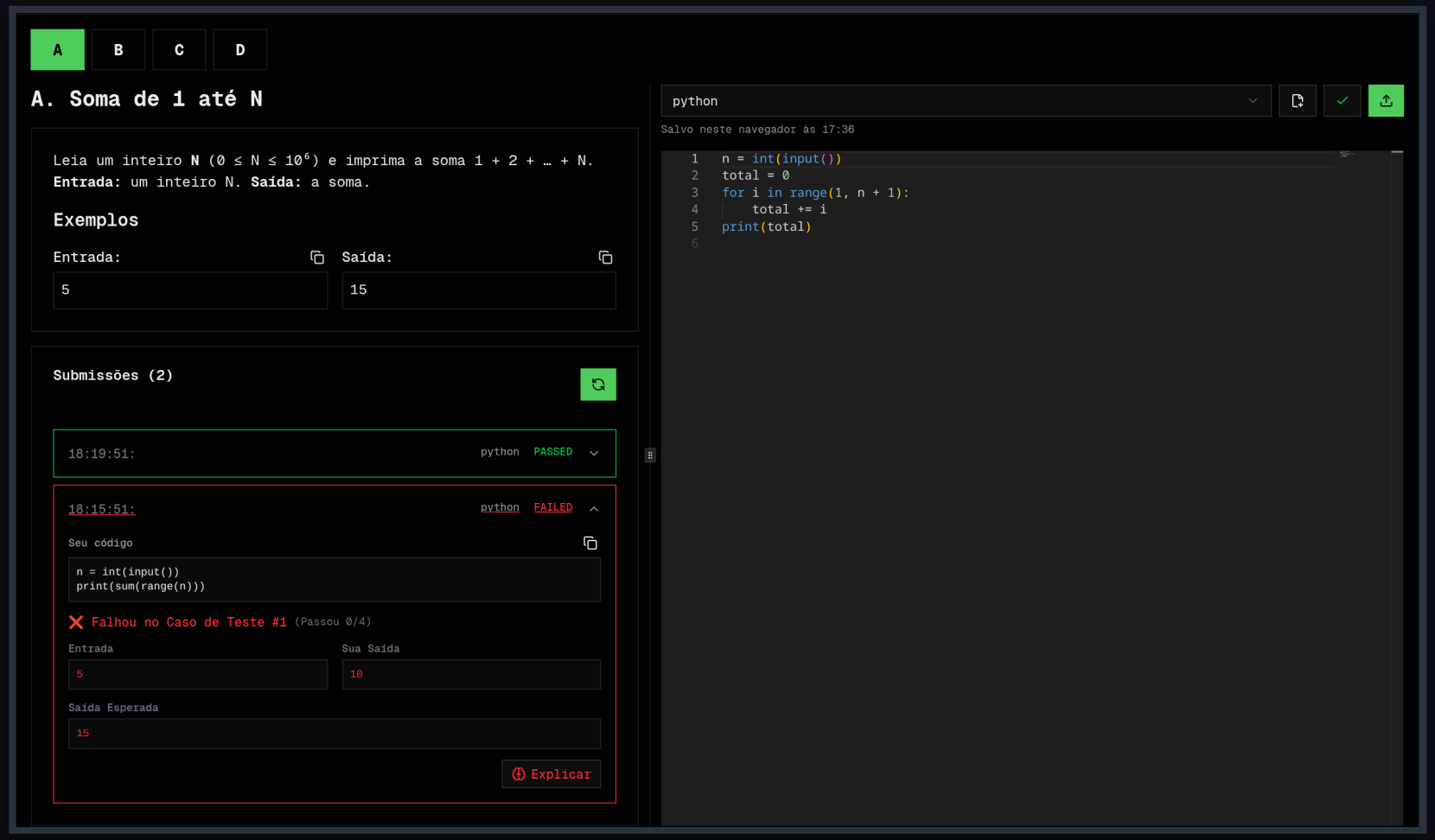
O editor salva sozinho no navegador



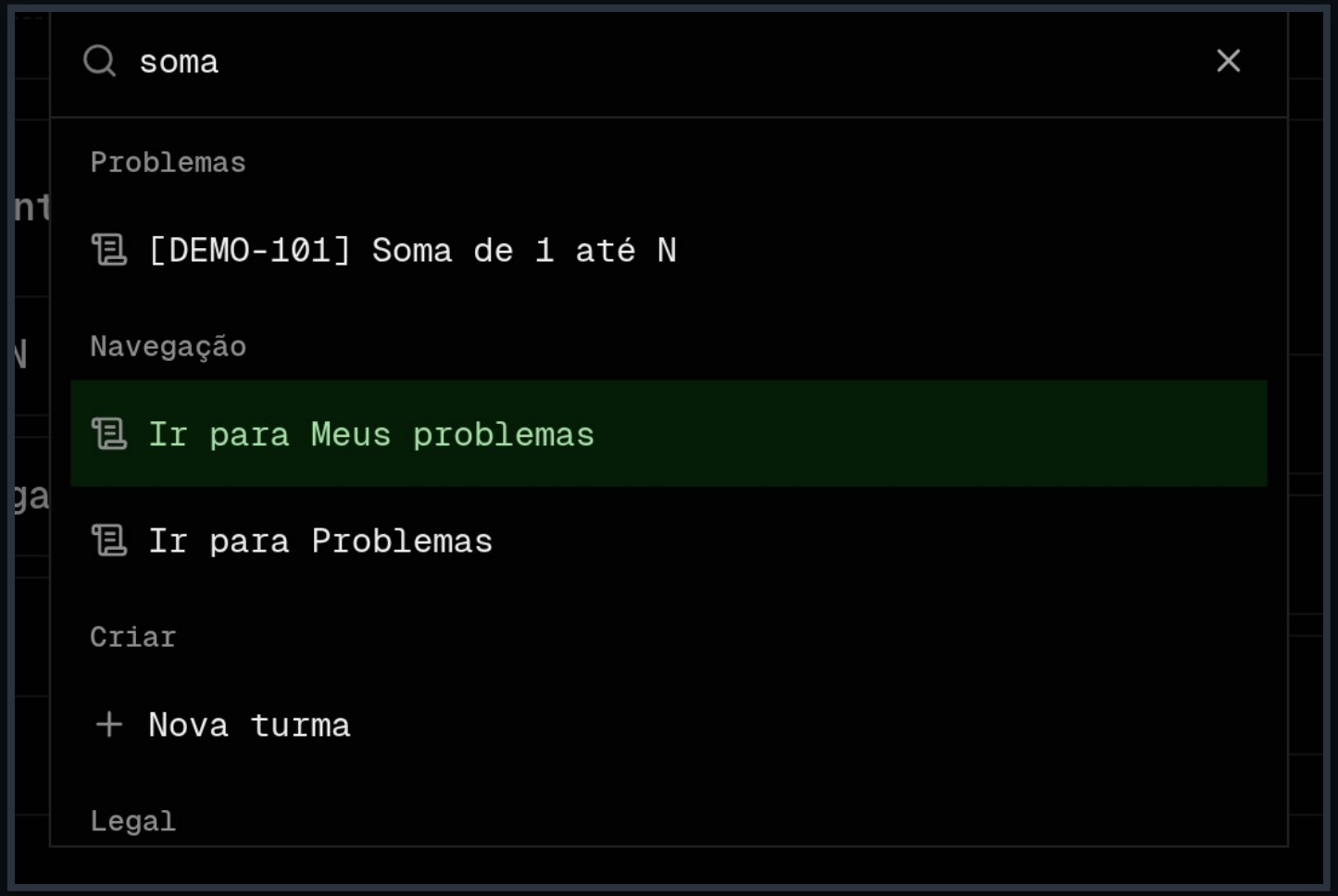
Cada envio guarda o código e pode pedir explicação



O caso que falhou mostra entrada, saída e esperado



Ctrl K abre uma busca que leva a qualquer lugar



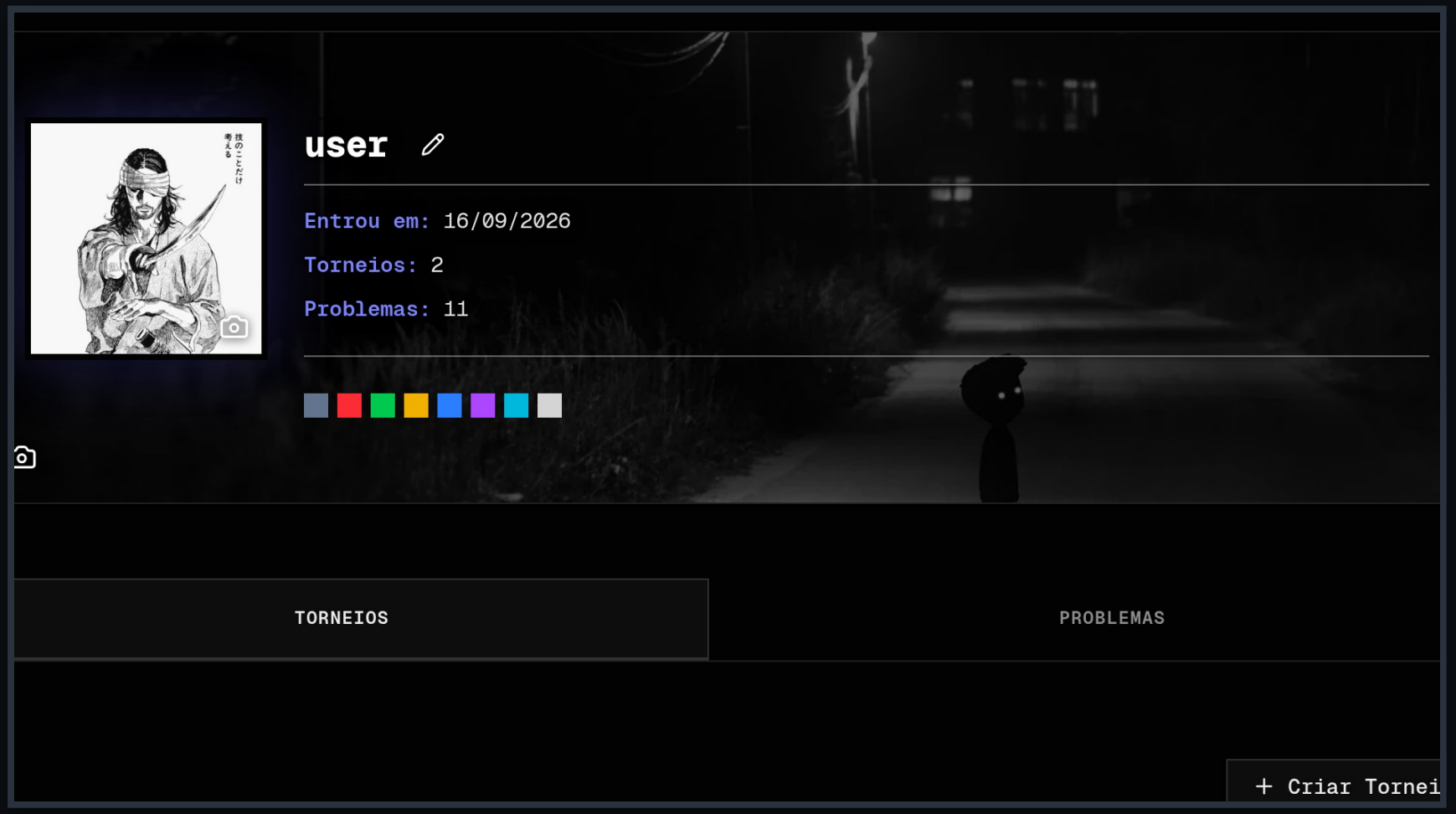
O atalho aparece no próprio menu



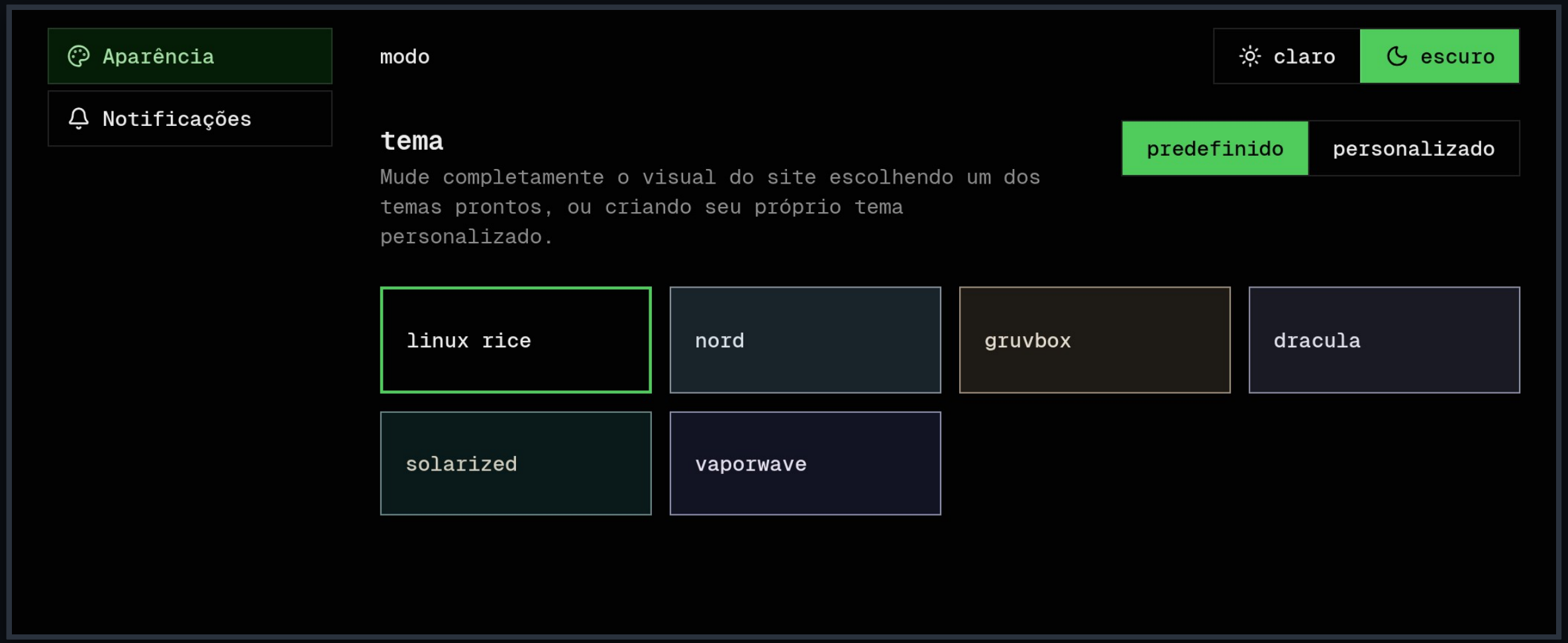
Notificações avisam o veredito e as aprovações



O perfil mostra os torneios e problemas de cada um



Seis temas, ou um tema próprio



Até a página de erro fala a língua do programador



A turma tem aulas com prazo e revisão

Aulas e turma

listas de exercícios do professor

O aluno entra na turma com um código de seis letras



As aulas da turma têm prazo

Aula 1 — Laços

Exercícios da semana.

2 exercícios | Prazo 18/10/2026

Aula 2 — Condicionais

Exercícios da semana.

2 exercícios | Prazo 21/10/2026

Aula 3 — Strings

Exercícios da semana.

1 exercícios | Prazo 24/10/2026

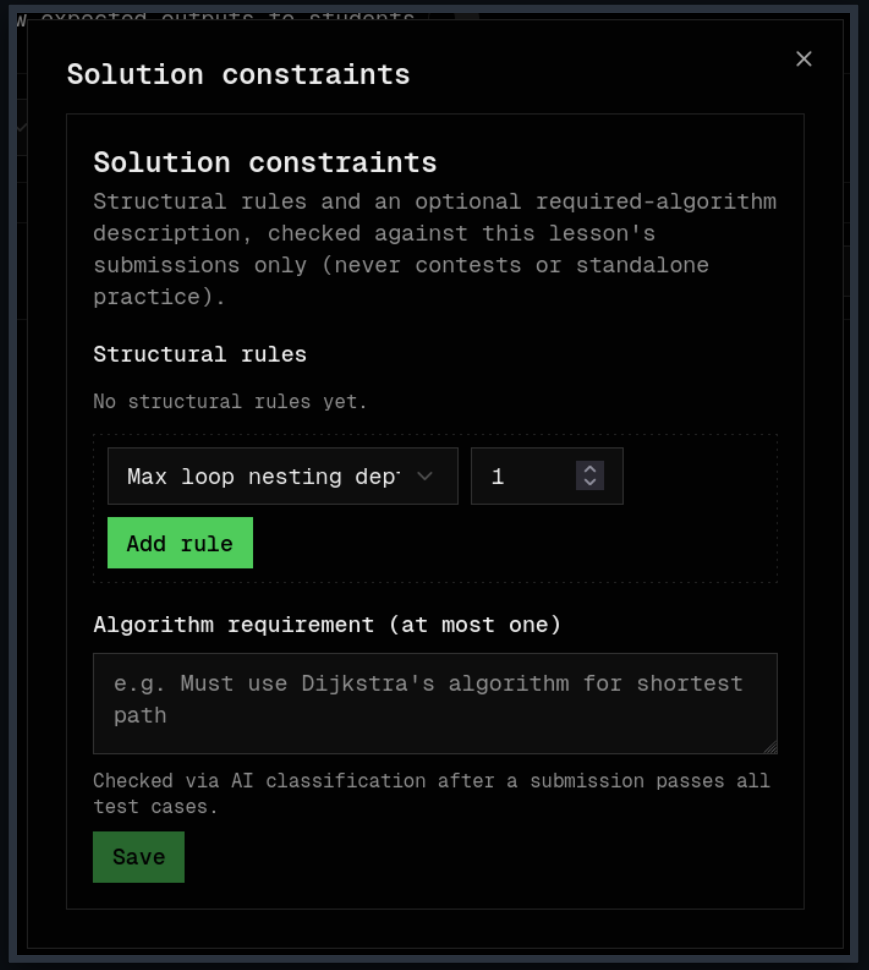
Blocos para quem nunca programou



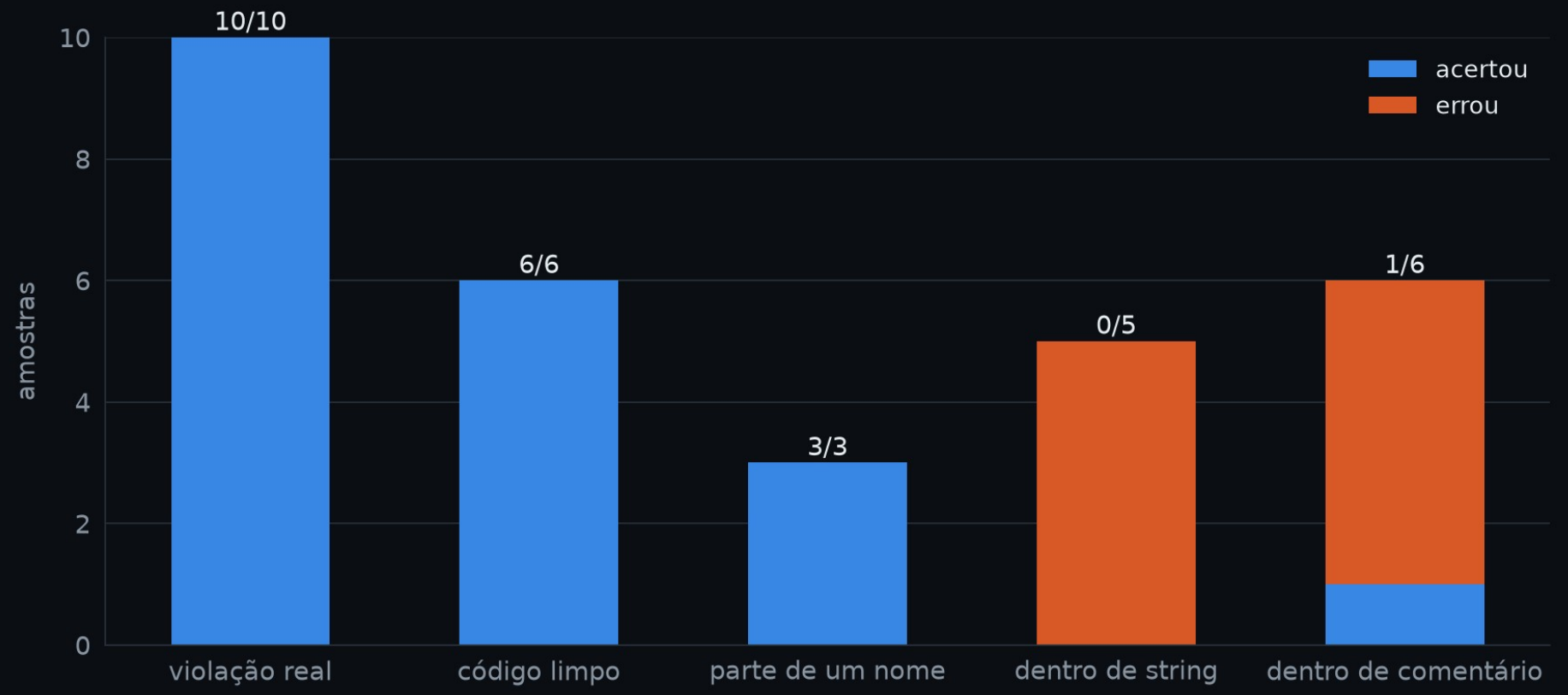
Parsons: ordenar as linhas certas



O professor exige como resolver, não só o resultado

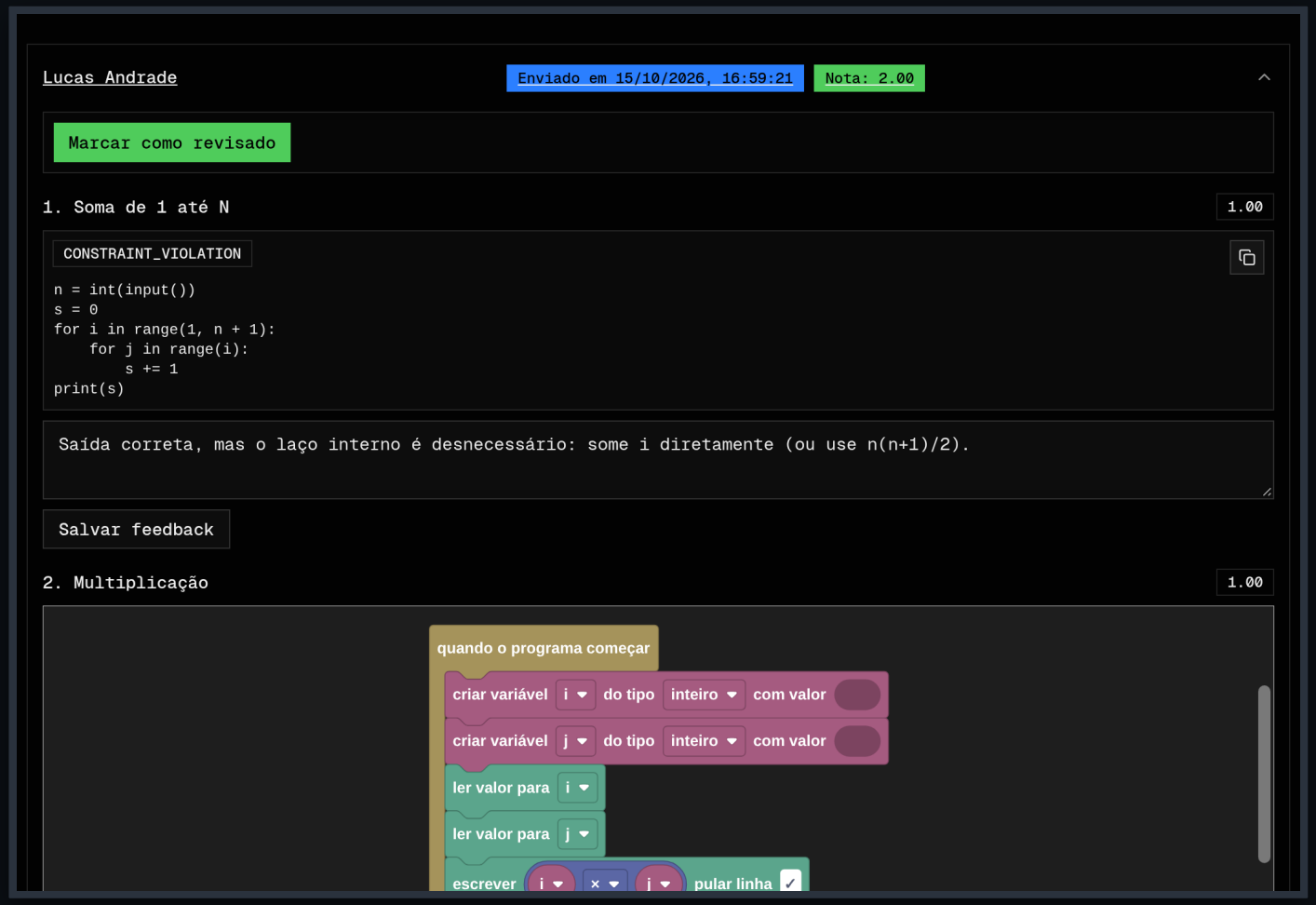


Restrições acertam código real e erram texto em strings e comentários



30 amostras rotuladas à mão

O professor revisa e comenta cada exercício



Notas da turma inteira numa tabela

Notas — Algoritmos I — Turma demo

Notas de cada aluno em cada tarefa publicada, a partir das entregas.

← Voltar para a turma | Exportar CSV

Cada exercício vale a fração de casos de teste aprovados (0-1); – indica tarefa não entregue.

Aluno	Aula 1 — Laços			Aula 2 — Condicionais			Aula 3 — Strings		Total
	Soma de 1 até…	Tabuada	Σ	Maior de três	Número primo	Σ	Contagem de v…	Σ	
Beatriz Lima	1.00	1.00	**2.00**	1.00	0.60	**1.60**	0.50	**0.50**	**4.10 / 5**
Camila Rocha	0.50	1.00	**1.50**	–	–	–	1.00	**1.00**	**2.50 / 5**
Lucas Andrade	0.75	1.00	**1.75**	1.00	1.00	**2.00**	1.00	**1.00**	**4.75 / 5**
Pedro Alves	–	–	–	1.00	0.80	**1.80**	0.75	**0.75**	**2.55 / 5**
Rafael Souza	1.00	0.50	**1.50**	0.75	1.00	**1.75**	–	–	**3.25 / 5**
user	1.00	0.00	**1.00**	0.50	1.00	**1.50**	1.00	**1.00**	**3.50 / 5**

O CSV abre certo no Excel e não executa fórmulas

`=HYPERLINK("http://…")` → `'=HYPERLINK("http://…")`

RFC 4180 | BOM para o Excel | fórmulas neutralizadas

O professor agora vê quem copiou de quem

Plágio

similaridade de código, estilo MOSS

O código vira impressões digitais antes da comparação



Estilo MOSS (Schleimer et al., 2003)

Renomear variáveis não engana a comparação

```
for (int i = 0; i < n; i++) total += v[i];
```



```
for ( int V = N ; V < V ; V ++ ) V += V [ V ] ;
```

Renomear e reformatar não muda o que é comparado.

O relatório lista os pares mais parecidos

← Voltar para a competição

Similaridade de código

Compare as respostas dos participantes a cada problema de Maratona de Algoritmos — demo

Problema: A. Soma de 1 até N

Comparar: Melhor submissão

Similaridade mínima: ≥ 50%

ⓘ As notas vêm de impressões digitais de tokens (estilo MOSS): identificadores, literais, comentários e formatação são ignorados, e o código que a maioria dos alunos compartilha é descontado. Uma nota alta é motivo para ler os dois programas, não prova de cópia.

Alunos	Linguagem	Similaridade	Coincidência (cada)
Beatriz Lima · Lucas Andrade	python	**100%**	100% / 100%

Lado a lado, os trechos iguais aparecem destacados

1 região coincidente · Mesma cor = mesmo bloco nos dois programas. Clique em um bloco para ir até o correspondente.

Beatriz Lima — 100% coincidente

```
# soma de 1 ate n
def acumula(limite):
    resultado = 0
    # percorre todos os numeros
    for k in range(1, limite + 1):
        resultado += k
    return resultado

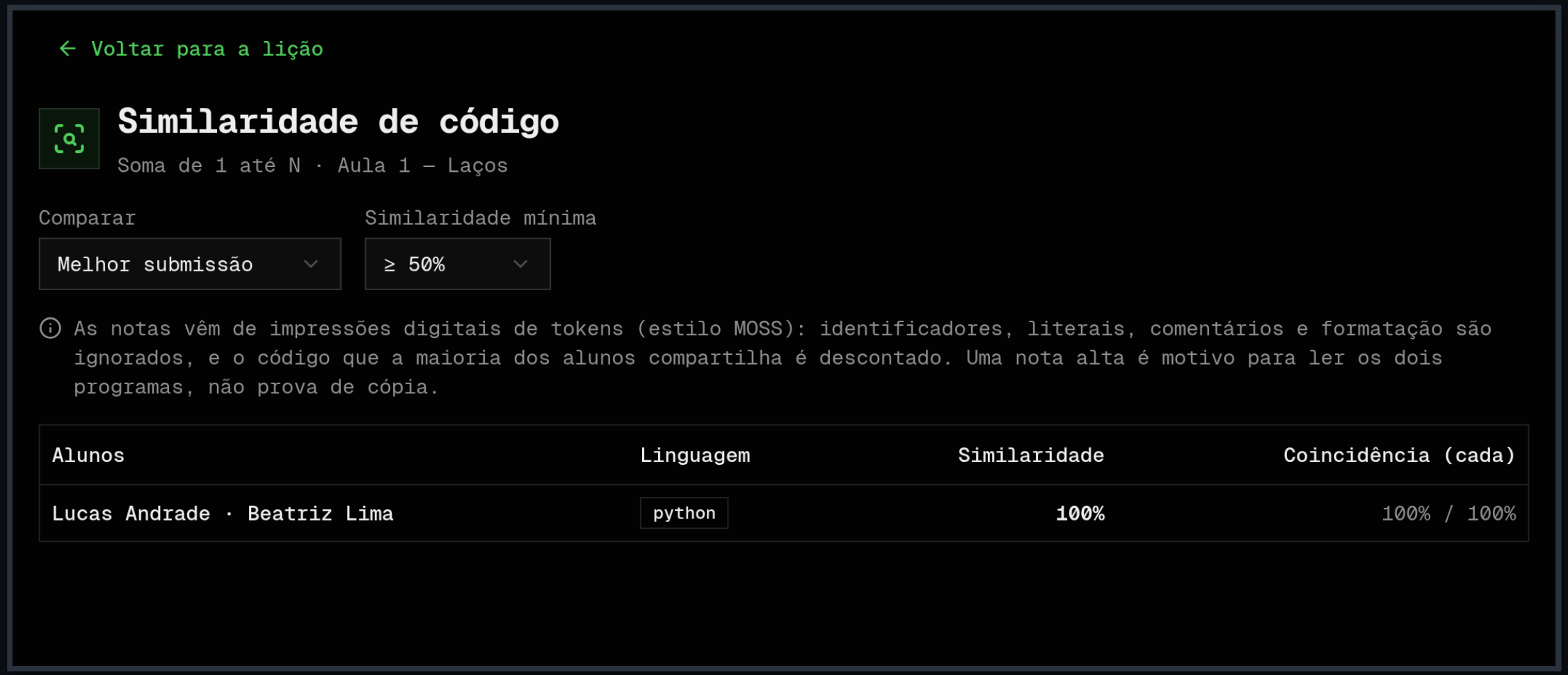

valor = int(input())
if valor < 1:
    print(0)
else:
    print(acumula(valor))
```

Lucas Andrade — 100% coincidente

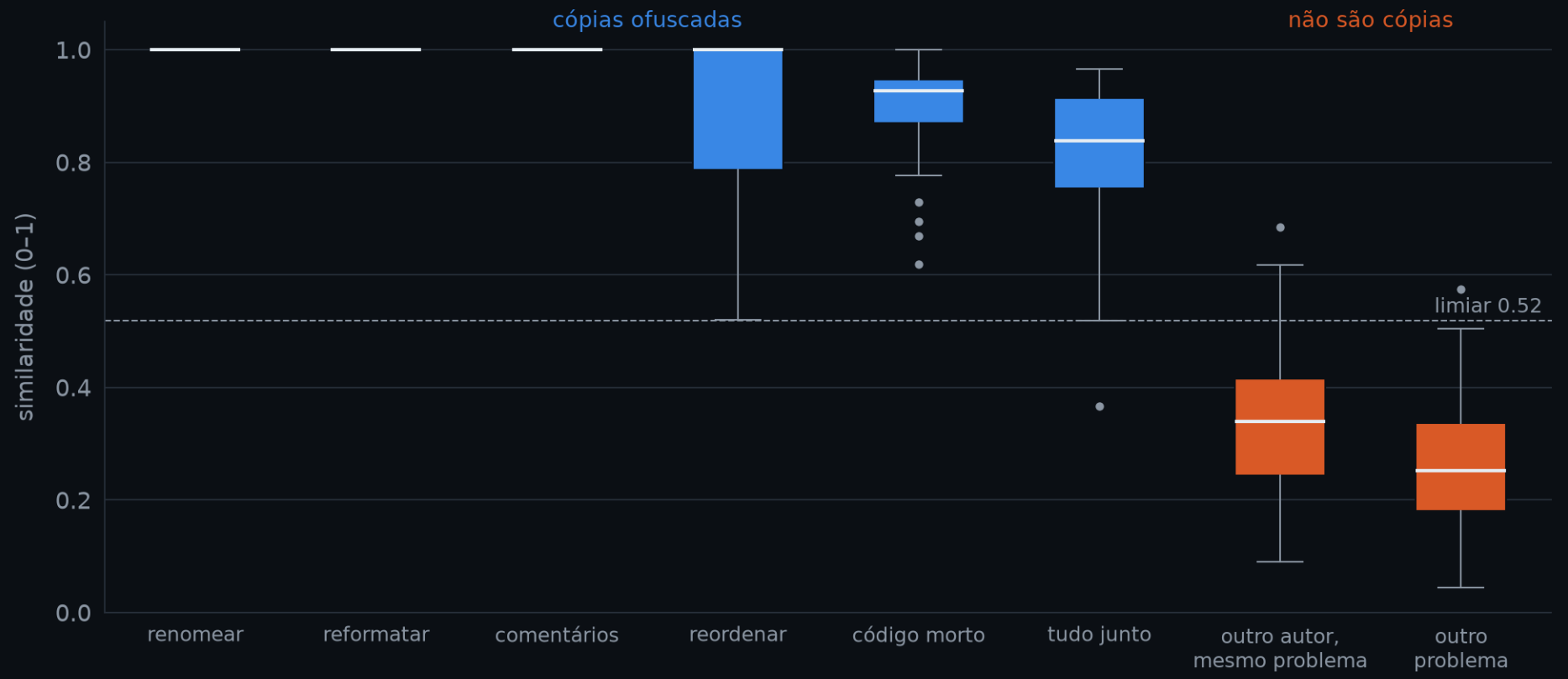
```
def soma_ate(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total


n = int(input())
if n < 1:
    print(0)
else:
    print(soma_ate(n))
```

A comparação também funciona dentro das aulas



Cópias ofuscadas ficam acima de 0,52; soluções independentes, abaixo



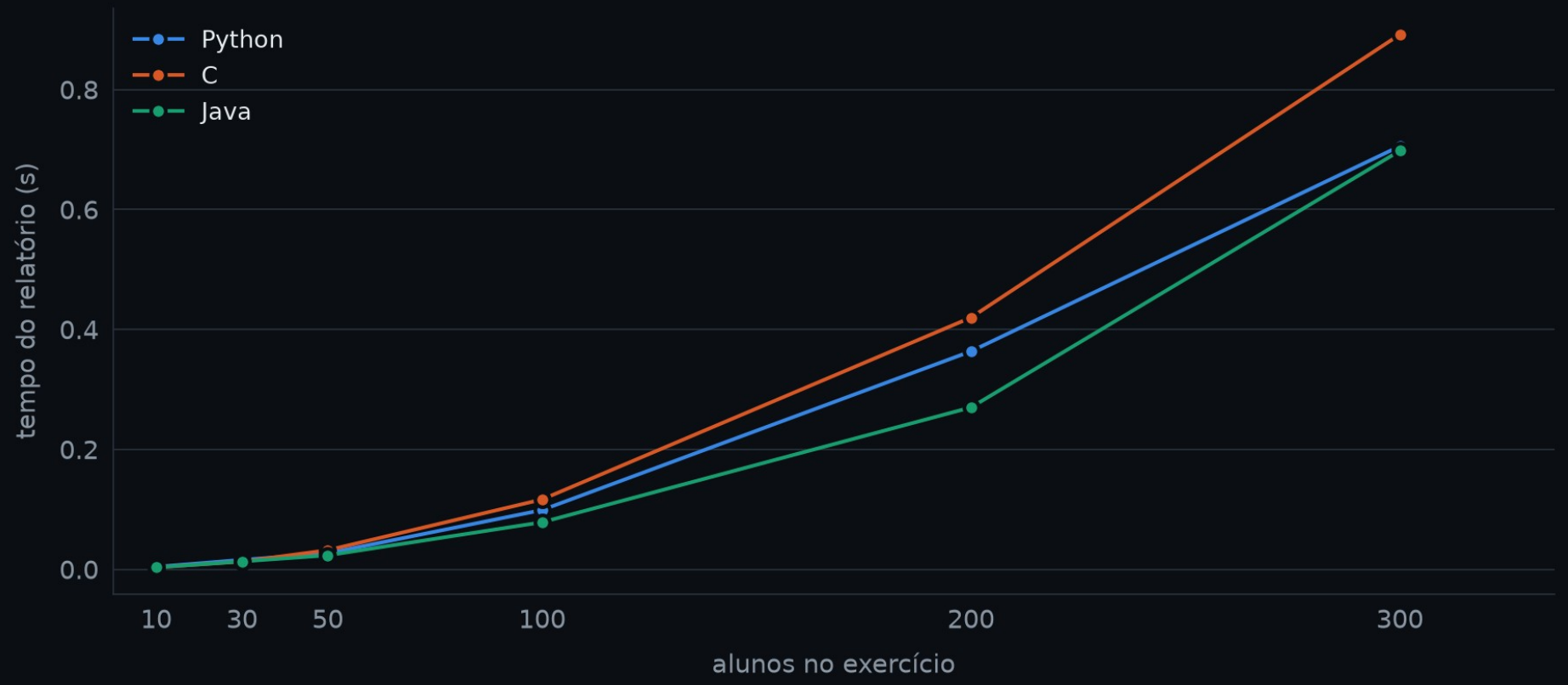
36 soluções originais, 6 ofuscações cada

Com o limiar de 0,52, 98% de precisão e 99,5% de revocação



216 cópias contra 198 pares independentes

200 alunos comparados em menos de meio segundo



Um exercício, uma linguagem, todos os pares

Plágio entre linguagens e código de IA passam despercebidos



Só compara a mesma linguagem; não detecta código gerado por IA.

Um teste informal já mudou o sistema

INFORMAL

5 colegas · Discord · sem roteiro

O que incomodou virou mudança

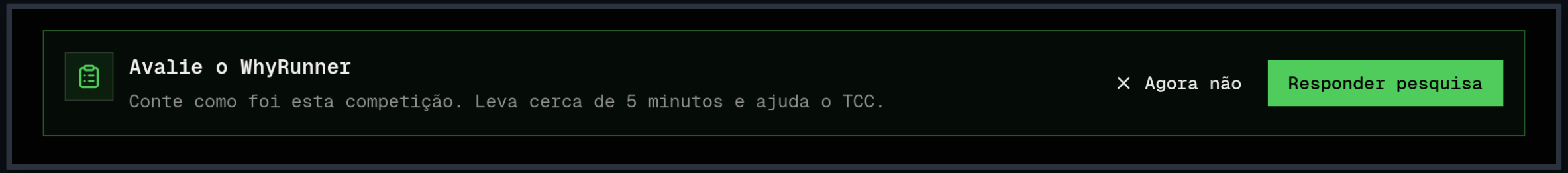
código oculto	→ **código visível**
medalhas	→ **placar ICPC**
JSON opaco	→ **erro com campo**

Gostaram do visual e do nome

tema escuro, estilo terminal

o nome: “Why” = “Uai”

A pesquisa aparece dentro do sistema, no fim da atividade



Quatro questionários, com termo de consentimento



A usabilidade é medida pelo SUS



Usabilidade geral

Escala de Usabilidade do Sistema (SUS). Responda pensando no WhyRunner como um todo.

1. Acho que gostaria de usar o WhyRunner com frequência.

1 2 3 4 5

1 = Discordo totalmente

5 = Concordo totalmente

2. Achei o WhyRunner desnecessariamente complexo.

1 2 3 4 5

1 = Discordo totalmente

5 = Concordo totalmente

3. Achei o WhyRunner fácil de usar.

Os resultados saem prontos, anônimos e em CSV



Ainda sem respostas

O primeiro teste em sala é em 29/10/2026

29/10/2026

- torneio
- lista de exercícios
- questionários + SUS

O que o sistema faz bem

sem fila externa

isolate da IOI

IA sob controle do professor

Limitação: ainda sem avaliação com alunos



A pergunta de pesquisa ainda não tem resposta.

Limitação: em produção, o judge tem uma CPU só



Na Fly.io, 1 vCPU: concorrência sem ganho.

Limitação: erros do Portugol viram resposta errada



O console do Portugol termina com código 0 mesmo com erro.

Limitação: memória excedida é heurística



Sem cgroups, não há sinal limpo de falta de memória.

Limitação: restrições leem texto, não a árvore do código



Palavras em strings e comentários disparam falso positivo.

Próximos passos depois do teste em sala

detectar código de IA

isolate com cgroups

judge multi-core

Próximas etapas, em ordem



Teste em sala

29/10

Análise

Defesa

Referências

ANTHROPIC. Claude Haiku 5.5. 2026. Disponível em: https://www.anthropic.com/claude.

BROOKE, J. SUS: a quick and dirty usability scale. Usability Evaluation in Industry, 1996.

CAMPOS, C. P.; FERREIRA, C. E. BOCA: um sistema de apoio a competições de programação. WEI, 2004.

CARNEIRO, M. F. Sistema web para auxílio no treinamento para maratonas de programação. UTFPR, 2012.

DRIZZLE TEAM. Drizzle ORM: TypeScript ORM for SQL. 2024.

GEMINI TEAM, GOOGLE. Gemini: a family of highly capable multimodal models. 2023.

LIMA, G. M. Gamma Online Judge: plataforma para as questões das maratonas UnB de programação. 2022.

MAREŠ, M.; BLACKHAM, B. A new contest sandbox. Olympiads in Informatics, v. 6, 2012.

MATSAKIS, N. D.; KLOCK, F. S. The Rust language. ACM SIGAda Ada Letters, 2014.

MERKEL, D. Docker: lightweight Linux containers for consistent development and deployment. Linux Journal, 2014.

RUN CODES. Run Codes: plataforma de programação online. 2024.

SCHLEIMER, S.; WILKERSON, D. S.; AIKEN, A. Winnowing: local algorithms for document fingerprinting. ACM SIGMOD, 2003.

SILVA, J. T. S. et al. Ataux: uma ferramenta para auxiliar o acompanhamento de submissões de questões de programação. 2021.

STONEBRAKER, M.; ROWE, L. A. The design of POSTGRES. ACM SIGMOD, 1986.

VERCEL INC. Next.js: the React framework for the web. 2024.

Obrigado!

Obrigado!

Perguntas?